An Overview of Compilation

Uday Khedker

(www.cse.iitb.ac.in/~uday)

Department of Computer Science and Engineering,

Indian Institute of Technology, Bombay

January 2026

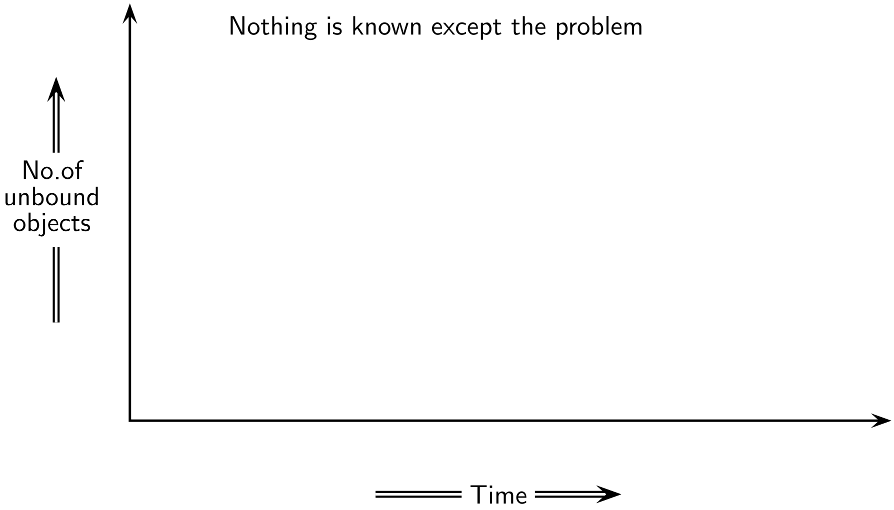
Outline

Outline

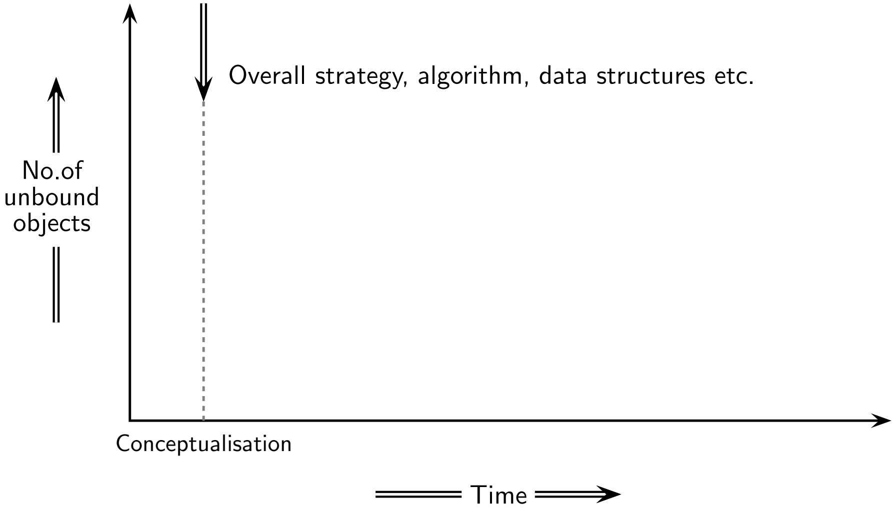
- The big picture
- Course plan
- Expectation management
- Compilation phases
- Compilation models
- CS306: Incremental construction of a compiler

The Big Picture

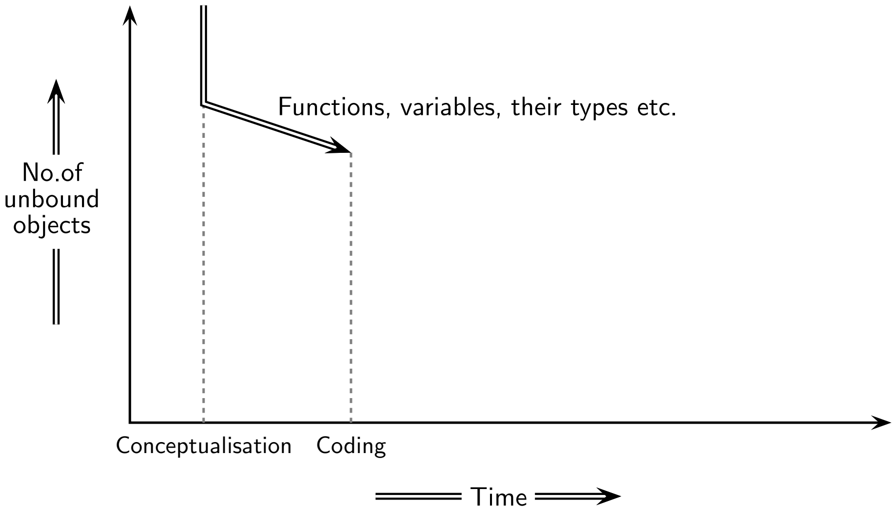
Binding


Nothing is known except the problem
No.of unbound objects
Time

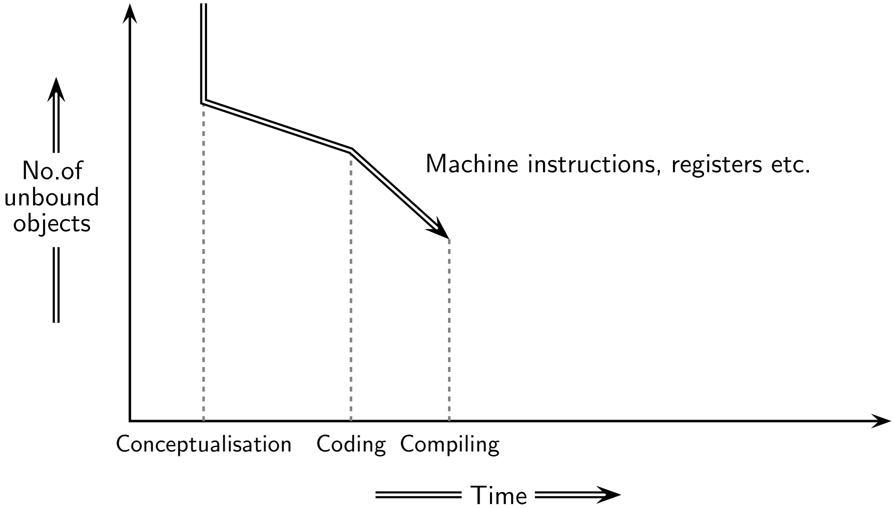
Binding


Overall strategy, algorithm, data structures etc.
No.of unbound objects
Conceptualisation
Time

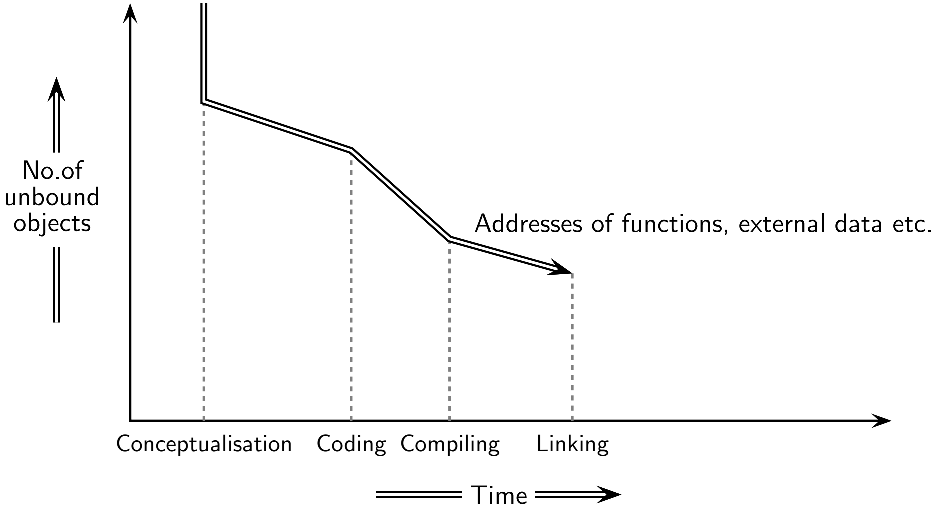
Binding


No.of unbound objects
Functions, variables, their types etc.
Conceptualisation
Coding
Time

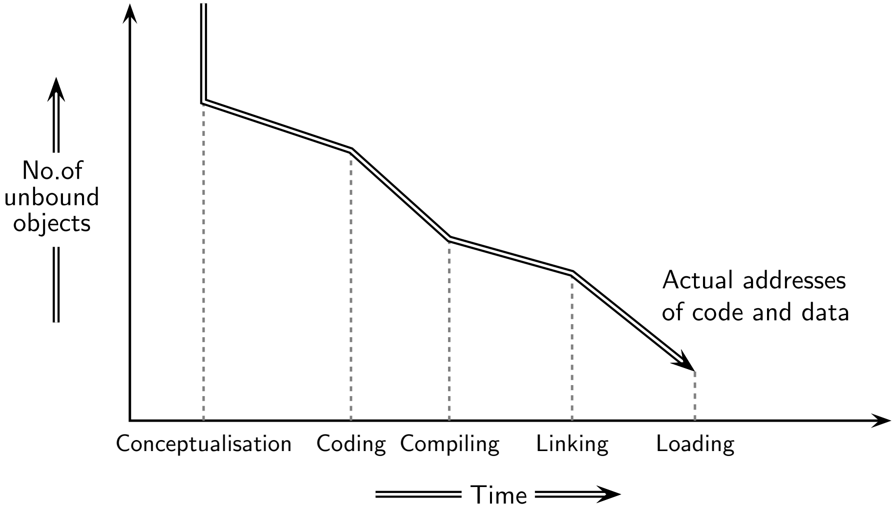
Binding


No.of unbound objects
Machine instructions, registers etc.
Conceptualisation
Coding
Compiling
Time

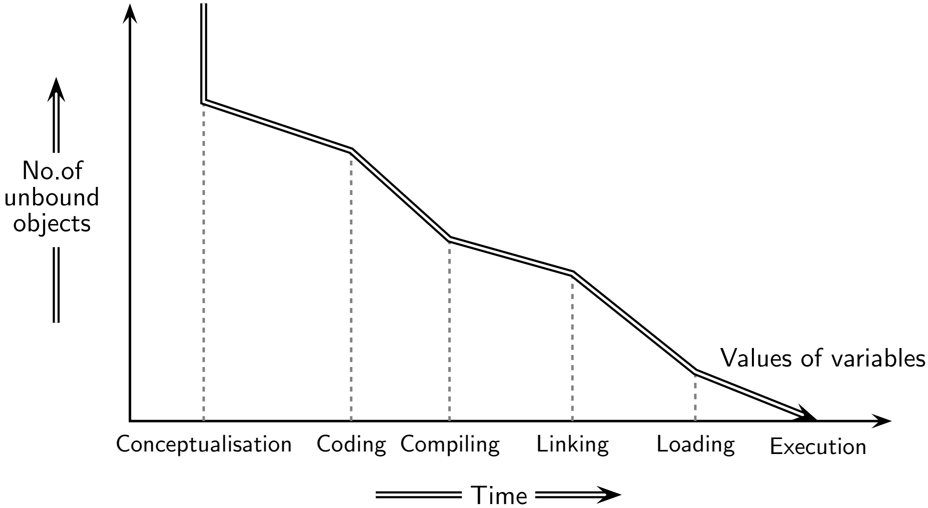
Binding


No.of unbound objects
Addresses of functions, external data etc.
Conceptualisation
Coding
Compiling
Linking
Time

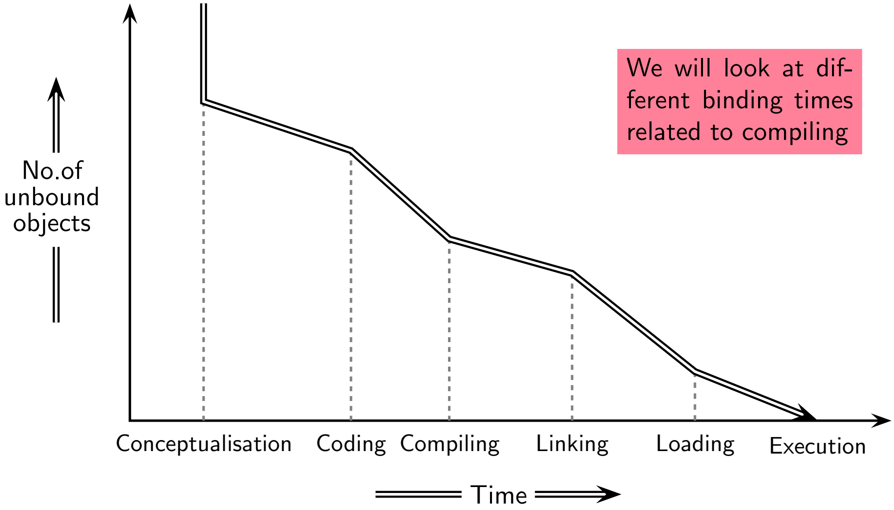
Binding


No.of unbound objects
Actual addresses of code and data
Conceptualisation
Coding
Compiling
Linking
Loading
Time

Binding


No.of unbound objects
Values of variables
Conceptualisation
Coding
Compiling
Linking
Loading
Execution
Time

Binding



We will look at different binding times related to compiling

Implementation Mechanisms

Source Program

↓

$\boxed{\text{Translator}}$

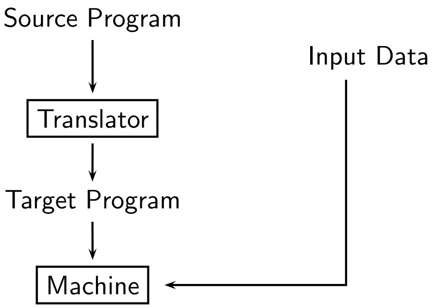
↓

Target Program

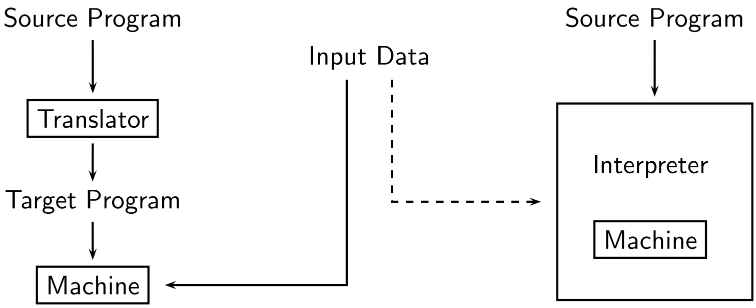
↓

$\boxed{\text{Machine}}$

Implementation Mechanisms



Implementation Mechanisms



Comparing the Implementation Mechanisms

$$\text{Translation} \quad = \quad \text{Analysis} + \text{Synthesis}$$
$$\text{Interpretation} \quad = \quad \text{Analysis} + \text{Execution}$$

Comparing the Implementation Mechanisms

$$\begin{aligned} \text{Translation} &= \text{Analysis} + \text{Synthesis} \\ \text{Interpretation} &= \text{Analysis} + \text{Execution} \end{aligned}$$

Implementation mechanism	Input	Output	Separate execution	Input for the input program
Translation	Program	Equivalent program	Required	Not required
Interpretation	Program	The result of the Program	Not required	Required

Seeing the Difference Between Compilation and Interpretation

```
$ ./lp --help
Usage: lp [OPTION...]
  -c                   Compile the input into three address code and
                       print it
  -i                   Interpret the input and print result
  -?, --help           Give this help list
      --usage          Give a short usage message

$ ./lp -i
a = 10 + 20 * 30;
> a = 610

$ ./lp -c
a = 10 + 20 * 30;
The three address code generated for the input is
t0 = 20 * 30
t1 = 10 + t0
a = t1
```

Implementation Mechanisms as "Bridges"

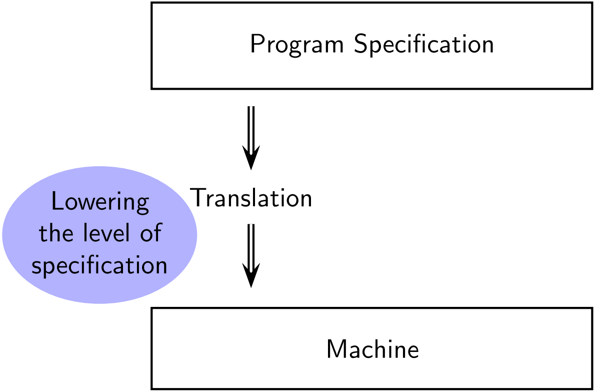
- "Gap" between the "levels" of program specification and execution

Program Specification

Machine

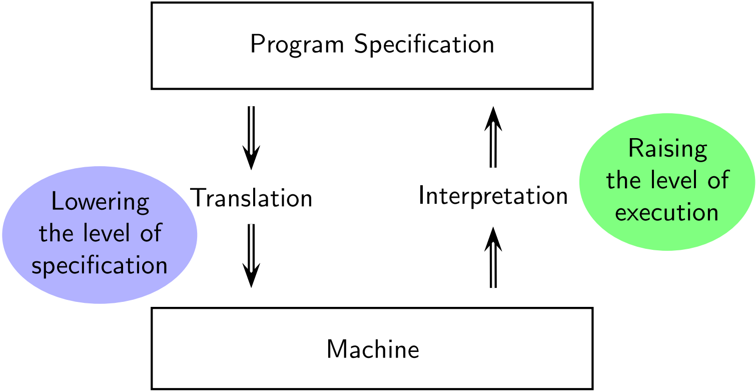
Implementation Mechanisms as "Bridges"

- "Gap" between the "levels" of program specification and execution



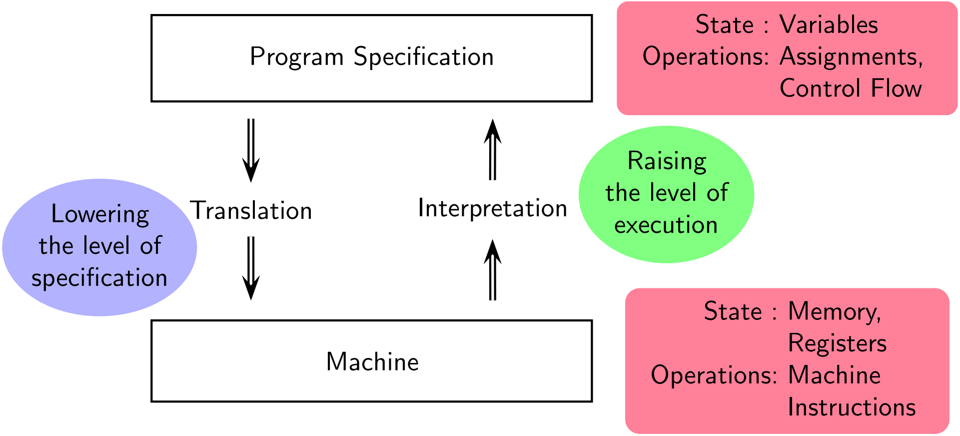
Implementation Mechanisms as "Bridges"

- "Gap" between the "levels" of program specification and execution



Implementation Mechanisms as "Bridges"

- "Gap" between the "levels" of program specification and execution



A Source Program in C++: High Level Abstraction

```
#include <iostream>
using namespace std;

int main()
{
    int n, fact=1;

    cout << "Enter the number: ";
    cin >> n;
    for (int i=n; i > 0; i--)
        fact = fact * i;

    cout << "The factorial of " << n << " is " << fact << endl;

    return 0;
}
```

Its Target Program: Low Level Abstraction (1)

f3 0f 1e fa 48 83 ec 08 48 8b 05 d9 2f 00 00 48 85 c0 74 02 ff d0 48 83 c4
08 c3 ff 35 5a 2f 00 00 f2 ff 25 5b 2f 00 00 0f 1f 00 f3 0f 1e fa 68 00 00
00 00 f2 e9 e1 ff ff ff 90 f3 0f 1e fa 68 01 00 00 00 f2 e9 d1 ff ff ff 90
f3 0f 1e fa 68 02 00 00 00 f2 e9 c1 ff ff ff 90 f3 0f 1e fa 68 03 00 00 00
f2 e9 b1 ff ff ff 90 f3 0f 1e fa 68 04 00 00 00 f2 e9 a1 ff ff ff 90 f3 0f
1e fa 68 05 00 00 00 f2 e9 91 ff ff ff 90 f3 0f 1e fa 68 06 00 00 00 f2 e9
81 ff ff ff 90 f3 0f 1e fa f2 ff 25 1d 2f 00 00 0f 1f 44 00 00 f3 0f 1e fa
f2 ff 25 d5 2e 00 00 0f 1f 44 00 00 f3 0f 1e fa f2 ff 25 cd 2e 00 00 0f 1f
44 00 00 f3 0f 1e fa f2 ff 25 c5 2e 00 00 0f 1f 44 00 00 f3 0f 1e fa f2 ff
25 bd 2e 00 00 0f 1f 44 00 00 f3 0f 1e fa f2 ff 25 b5 2e 00 00 0f 1f 44 00
00 f3 0f 1e fa f2 ff 25 ad 2e 00 00 0f 1f 44 00 00 f3 0f 1e fa f2 ff 25 a5
2e 00 00 0f 1f 44 00 00 f3 0f 1e fa 31 ed 49 89 d1 5e 48 89 e2 48 83 e4 f0
50 54 4c 8d 05 86 02 00 00 48 8d 0d 0f 02 00 00 48 8d 3d c1 00 00 00 ff 15
92 2e 00 00 f4 90 48 8d 3d b9 2e 00 00 48 8d 05 b2 2e 00 00 48 39 f8 74 15
48 8b 05 6e 2e 00 00 48 85 c0 74 09 ff e0 0f 1f 80 00 00 00 00 c3 0f 1f 80
00 00 00 00 48 8d 3d 89 2e 00 00 48 8d 35 82 2e 00 00 48 29 fe 48 89 f0 48
c1 ee 3f 48 c1 f8 03 48 01 c6 48 d1 fe 74 14 48 8b 05 45 2e 00 00 48 85 c0
74 08 ff e0 66 0f 1f 44 00 00 c3 0f 1f 80 00 00 00 00 f3 0f 1e fa 80 3d ad
30 00 00 00 75 2b 55 48 83 3d f2 2d 00 00 00 48 89 e5 74 0c 48 8b 3d 26 2e
00 00 e8 b9 fe ff ff e8 64 ff ff ff c6 05 85 30 00 00 01 5d c3 0f 1f 00 c3

Its Target Program: Low Level Abstraction (2)

```
0f 1f 80 00 00 ff ff e8 64 ff ff ff c6 05 85 30 00 00 01 5d c3 0f 1f 00 c3
0f 1f 80 00 00 00 00 f3 0f 1e fa e9 77 ff ff ff f3 0f 1e fa 55 48 89 e5 48
83 ec 20 64 48 8b 04 25 28 00 00 00 48 89 45 f8 31 c0 c7 45 f0 01 00 00 00
48 8d 35 d3 0d 00 00 48 8d 3d 07 2e 00 00 e8 92 fe ff ff 48 8d 45 ec 48 89
c6 48 8d 3d 14 2f 00 00 e8 5f fe ff ff 8b 45 ec 89 45 f4 83 7d f4 00 7e 10
8b 45 f0 0f af 45 f4 89 45 f0 83 6d f4 01 eb ea 48 8d 35 a4 0d 00 00 48 8d
3d c5 2d 00 00 e8 50 fe ff ff 48 89 c2 8b 45 ec 89 c6 48 89 d7 e8 80 fe ff
ff 48 8d 35 93 0d 00 00 48 89 c7 e8 31 fe ff ff 48 89 c2 8b 45 f0 89 c6 48
89 d7 e8 61 fe ff ff 48 89 c2 48 8b 05 17 2d 00 00 48 89 c6 48 89 d7 e8 1c
fe ff ff b8 00 00 00 00 48 8b 4d f8 64 48 33 0c 25 28 00 00 00 74 05 e8 13
fe ff ff c9 c3 f3 0f 1e fa 55 48 89 e5 48 83 ec 10 89 7d fc 89 75 f8 83 7d
fc 01 75 32 81 7d f8 ff ff 00 00 75 29 48 8d 3d 72 2f 00 00 e8 f4 fd ff ff
48 8d 15 f5 2c 00 00 48 8d 35 5f 2f 00 00 48 8b 05 d7 2c 00 00 48 89 c7 e8
97 fd ff ff 90 c9 c3 f3 0f 1e fa 55 48 89 e5 be ff ff 00 00 bf 01 00 00 00
e8 9c ff ff ff 5d c3 66 2e 0f 1f 84 00 00 00 00 00 90 f3 0f 1e fa 41 57 4c
8d 3d 03 2a 00 00 41 56 49 89 d6 41 55 49 89 f5 41 54 41 89 fc 55 48 8d 2d
fc 29 00 00 53 4c 29 fd 48 83 ec 08 e8 7f fc ff ff 48 c1 fd 03 74 1f 31 db
0f 1f 80 00 00 00 00 4c 89 f2 4c 89 ee 44 89 e7 41 ff 14 df 48 83 c3 01 48
39 dd 75 ea 48 83 c4 08 5b 5d 41 5c 41 5d 41 5e 41 5f c3 66 66 2e 0f 1f 84
00 00 00 00 00 f3 0f 1e fa c3 f3 0f 1e fa 48 83 ec 08 48 83 c4 08 c3
```

Commands to Obtain the Low Level Abstraction

- Write the program and name the file `fact-iterative.cc`
- `g++ fact-iterative.cc` produces the executable in `a.out` file
- `strip a.out` removes names from the executable `a.out`
- `file a.out` produces the following output

  ```
  a.out:  ELF 64-bit LSB shared object, x86-64, version 1 (SYSV),
  dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2,
  BuildID[sha1]=0c218bf025a20bc43339dfd15cec41adc1c13946, for
  GNU/Linux 3.2.0, stripped
  ```
- `objdump -d a.out` produces the hexadecimal form along with assembly program
- `objdump` demo

Executable File Formats

- Unix, Linux, BSD, Solaris.
  - ELF (Executable and Linkable Format) (most common)
  - COFF, a.out, ECOFF (older formats)
- Mac OS. Mach-O (Mach Object)
- Windows. PE (Portable Executable)
- JVM. Byte code, JAR
- Browsers. WASM (Web Assembly)

Linking and Loading

- Linking: Resolving external references
  - An object file corresponds to a translation unit
  - May use data and functions from other object files
  - Library functions
    Static linking, dynamic linking, linking a shared object file
- Loading: Copying the program on the memory area specified by the OS
- How does the execution start for the command?

```
$ a.out
```

Linking and Loading

- Linking: Resolving external references
  - An object file corresponds to a translation unit
  - May use data and functions from other object files
  - Library functions
    Static linking, dynamic linking, linking a shared object file
- Loading: Copying the program on the memory area specified by the OS
- How does the execution start for the command?

```
$ a.out
```

  - The shell interpreter is the running process
  - It forks itself and execs the program `a.out`

Executable and Linkable Format

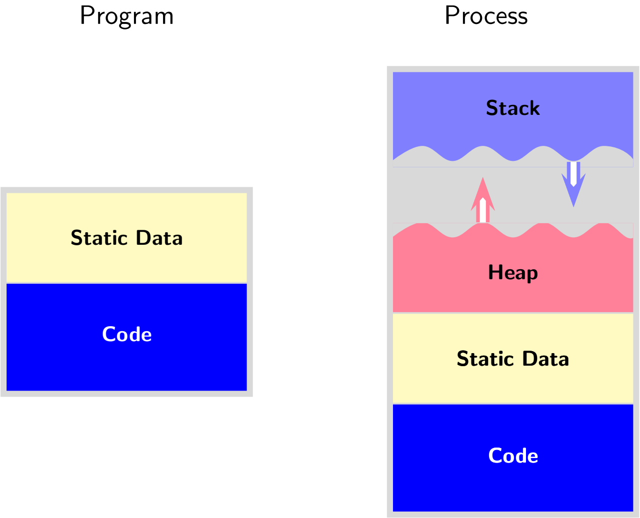
- Specifies
  - Code and data layout in the memory
  - Linking information
  - Loading information
- Key components
  - ELF Header. Identifies the file and architecture
  - Program Headers. How the OS should load the file
  - Section Headers. Used by linkers and debuggers
  - Sections. `.text`, `.data`, `.bss`, `.rodata`, `.symtab` etc.
- Significance
  - Enables dynamic linking
  - Supports position-independent code
  - Allows tools such as `ld`, `objdump`, `readelf`, `gdb` to work consistently
  - Portable across architectures (x86, ARM, RISC-V, etc.)
- `readelf` demo

Binary Files Using ELF

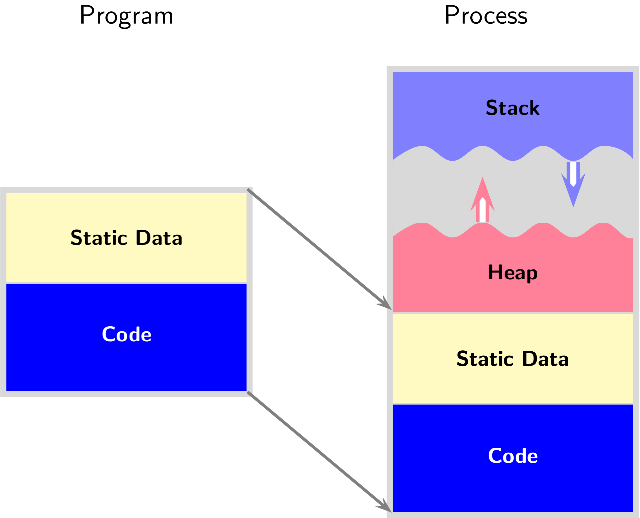
	`.o` (object)	`.so` (shared object)	`a.out` (executable)
Fully linked?	No	Yes	Yes
Used by linker?	Yes	Sometimes	No
Loaded by OS?	No	Yes	Yes
Shared at runtime?	No	Yes	No

A `.a` file is a searchable bundle of multiple object files in `ar` (archive) format

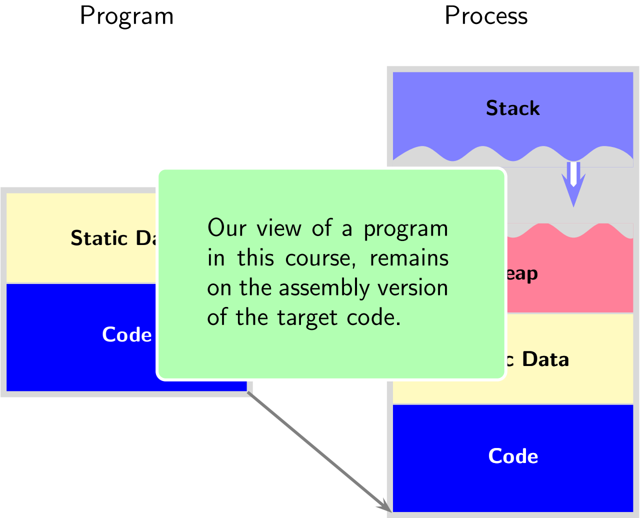
Program and Process



Program and Process



Program and Process



Our view of a program in this course, remains on the assembly version of the target code.

High and Low Level Abstractions: Our View

Input C statement

```
a = b<10?b:c+5;
```

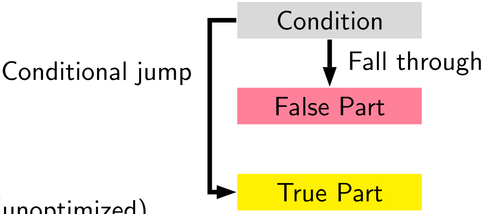
Spim assembly equivalent (unoptimized)

```
    lw     $v0, 4($fp)   ;     v0 <- b            # Is b smaller
    slti   $t1, $v0, 10  ;     t1 <- v0 < 10      # than 10?
    xori   $t2, $t1, 1   ;     t2 <- !t1
    bgtz   $t2, L0       ;     if t2 > 0 goto L0
    lw     $t3, 4($fp)   ;     t3 <- b            # YES
    b      L1            ;     goto L1
L0: lw     $t4, 8($fp)   ;L0: t4 <- c            # NO
    addi   $t3, $t4, 5   ;     t3 <- t4 + 5       # NO
L1: sw     0($fp), $t3   ;L1: a <- t3
```

High and Low Level Abstractions: Our View

Input C statement

```
a = b<10?b:c+5;
```



Spim assembly equivalent (unoptimized)

```
     lw     $v0, 4($fp)   ;     v0 <- b              # Is b smaller
     slti   $t1, $v0, 10  ;     t1 <- v0 < 10        # than 10?
     xori   $t2, $t1, 1   ;     t2 <- !t1
     bgtz   $t2, L0       ;     if t2 > 0 goto L0
     lw     $t3, 4($fp)   ;     t3 <- b              # YES
     b      L1            ;     goto L1
L0:  lw     $t4, 8($fp)   ;L0:  t4 <- c              # NO
     addi   $t3, $t4, 5   ;     t3 <- t4 + 5         # NO
L1:  sw     0($fp), $t3   ;L1:  a <- t3
```

High and Low Level Abstractions: Our View

NOT Condition

True Part

False Part

Input C statement

```
a = b<10?b:c+5;
```

Spim assembly equivalent (unoptimized)

```
    lw    $v0, 4($fp)   ;    v0 <- b          # Is b smaller
    slti  $t1, $v0, 10  ;    t1 <- v0 < 10    # than 10?
    xori  $t2, $t1, 1   ;    t2 <- !t1
    bgtz  $t2, L0       ;    if t2 > 0 goto L0
    lw    $t3, 4($fp)   ;    t3 <- b          # YES
    b     L1            ;    goto L1
L0: lw    $t4, 8($fp)   ;L0: t4 <- c          # NO
    addi  $t3, $t4, 5   ;    t3 <- t4 + 5     # NO
L1: sw    0($fp), $t3   ;L1: a <- t3
```

High and Low Level Abstractions: Our View

Input C statement

```
a = b<10?b:c+5;
```

Conditional jump

NOT Condition

Fall through

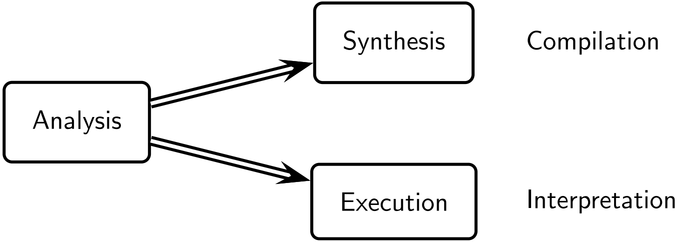
True Part

False Part

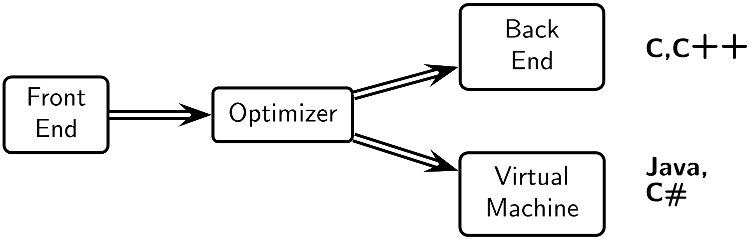
Spim assembly equivalent (unoptimized)

```
    lw    $v0, 4($fp)   ;     v0 <- b              # Is b smaller
    slti  $t1, $v0, 10  ;     t1 <- v0 < 10        # than 10?
    xori  $t2, $t1, 1   ;     t2 <- !t1
    bgtz  $t2, L0       ;     if t2 > 0 goto L0
    lw    $t3, 4($fp)   ;     t3 <- b              # YES
    b     L1            ;     goto L1
L0: lw    $t4, 8($fp)   ;L0:  t4 <- c              # NO
    addi  $t3, $t4, 5   ;     t3 <- t4 + 5         # NO
L1: sw    0($fp), $t3   ;L1:  a <- t3
```

Language Implementation Models



Language Processor Models



Why Do We Need Compilers and Interpreters, Both?

t: Time
A: Analysis, O: Optimization, S: Synthesis, E: Execution, B: Bookkeeping
p: Program, c: Compiler usage, i: Interpreter usage, j: Number of executions

$$t_c(p,j) = t_c^A(p) + t_c^O(p) + t_c^S(p) + \left(t_c^E(p) \times j\right)$$

$$t_i(p,j) = \left(t_i^A(p) + t_i^B(p) + t_i^E(p)\right) \times j$$

Why Do We Need Compilers and Interpreters, Both?

t: Time

A: Analysis, O: Optimization, S: Synthesis, E: Execution, B: Bookkeeping

p: Program, c: Compiler usage, i: Interpreter usage, j: Number of executions

compilation overheads

$$t_c(p,j) = \boxed{t_c^A(p) + t_c^O(p) + t_c^S(p)} + \left(t_c^E(p) \times j\right)$$

$$t_i(p,j) = \left(\boxed{t_i^A(p) + t_i^B(p)} + t_i^E(p)\right) \boxed{\times j}$$

interpretation overheads

Why Do We Need Compilers and Interpreters, Both?

t: Time
A: Analysis, O: Optimization, S: Synthesis, E: Execution, B: Bookkeeping
p: Program, c: Compiler usage, i: Interpreter usage, j: Number of executions

$$t_c(p,j) = \underbrace{t_c^A(p) + t_c^O(p) + t_c^S(p)}_{\text{compilation overheads}} + \left(t_c^E(p) \times j\right)$$

$$t_i(p,j) = \left(\underbrace{t_i^A(p) + t_i^B(p)}_{\text{interpretation overheads}} + t_i^E(p)\right) \times j$$

In general

- For large values of j, $t_c(p,j) \ll t_i(p,j)$
  Overheads of compilation are amortized over multiple executions

Why Do We Need Compilers and Interpreters, Both?

t: Time
A: Analysis, O: Optimization, S: Synthesis, E: Execution, B: Bookkeeping
p: Program, c: Compiler usage, i: Interpreter usage, j: Number of executions

compilation overheads

$$t_c(p,j) = \boxed{t_c^A(p) + t_c^O(p) + t_c^S(p)} + \left(t_c^E(p) \times j\right)$$

$$t_i(p,j) = \left(\boxed{t_i^A(p) + t_i^B(p)} + t_i^E(p)\right) \boxed{\times j}$$

interpretation overheads

In general

- For large values of j, $t_c(p,j) \ll t_i(p,j)$
  Overheads of compilation are amortized over multiple executions
- For small values of j, $t_c(p,j) \gg t_i(p,j)$
  Overheads of interpretations are meaningful for infrequently executed jobs

Why Do We Need Compilers and Interpreters, Both?

t: Time
A: Analysis, O: Optimization, S: Synthesis, E: Execution, B: Bookkeeping
p: Program, c: Compiler usage, i: Interpreter usage, j: Number of executions

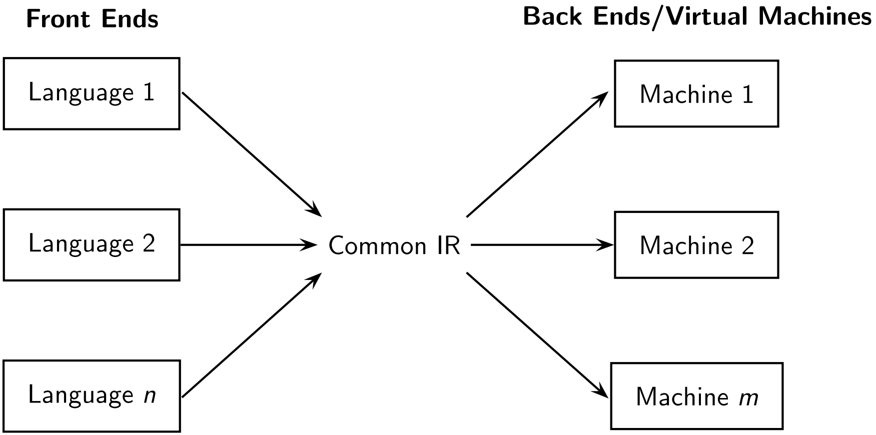
$$t_c(p,j) = \overbrace{t_c^A(p) + t_c^O(p) + t_c^S(p)}^{\text{compilation overheads}} + \left(t_c^E(p) \times j\right)$$

$$t_i(p,j) = \left(\underbrace{t_i^A(p) + t_i^B(p)}_{\text{interpretation overheads}} + t_i^E(p)\right) \times j$$

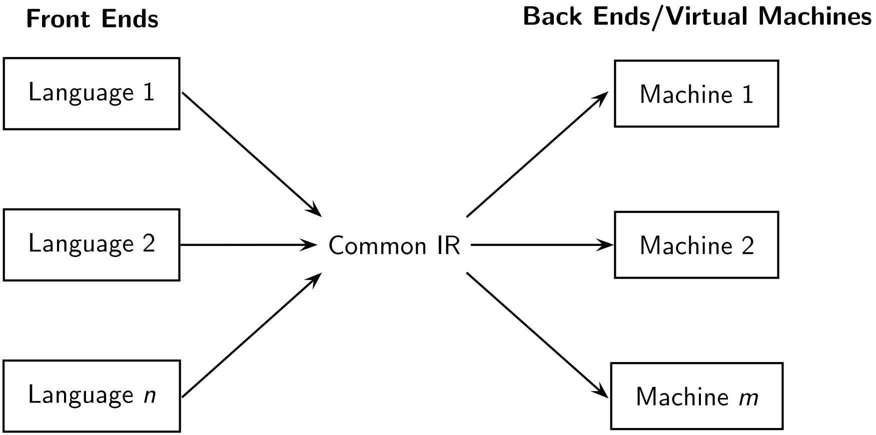
In general

- For large values of j, $t_c(p,j) \ll t_i(p,j)$
  Overheads of compilation are amortized over multiple executions
- For small values of j, $t_c(p,j) \gg t_i(p,j)$
  Overheads of interpretations are meaningful for infrequently executed jobs
- For any value of $j > 0$, $\left(t_c^E(p) \times j\right) \ll t_i(p,j)$
  Unless interpreter identifies hot paths/procedures and compiles them with run time data

Reusability of Language Processor Modules

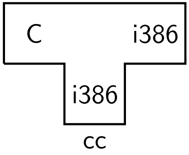


Reusability of Language Processor Modules



$m \times n$ compilers can be obtained from $m + n$ modules

Explaining Bootstrapping: T Notation for a Compiler



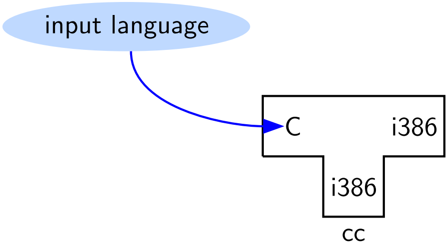
IIT Bombay
cs302: Implementation of Programming Languages

Topic:

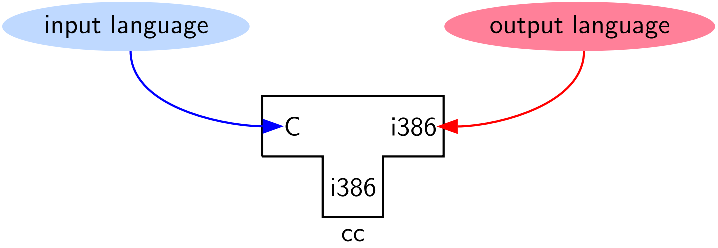
Compilation Overview

Section:

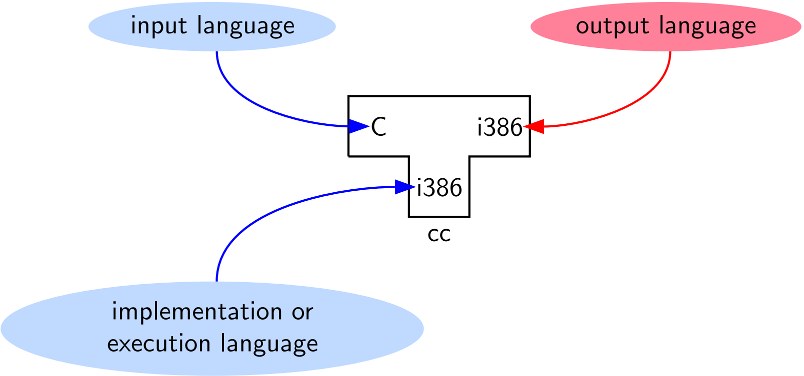
Explaining Bootstrapping: T Notation for a Compiler



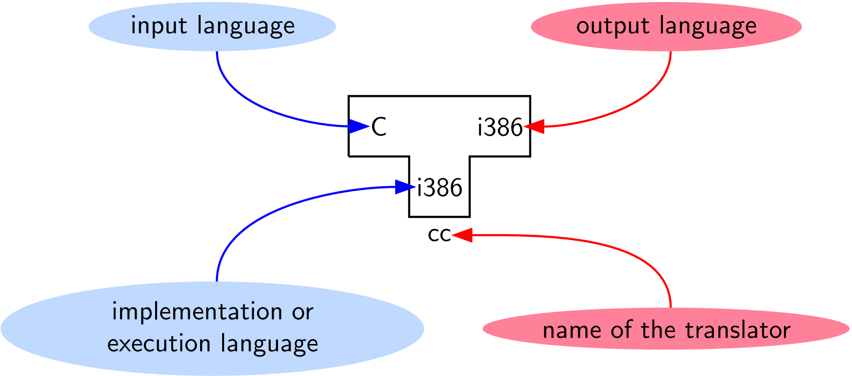
Explaining Bootstrapping: T Notation for a Compiler



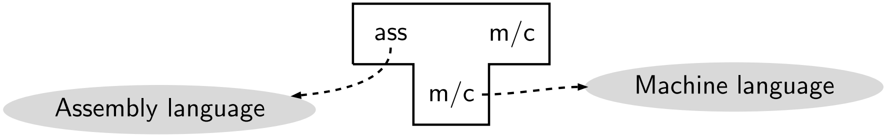
Explaining Bootstrapping: T Notation for a Compiler



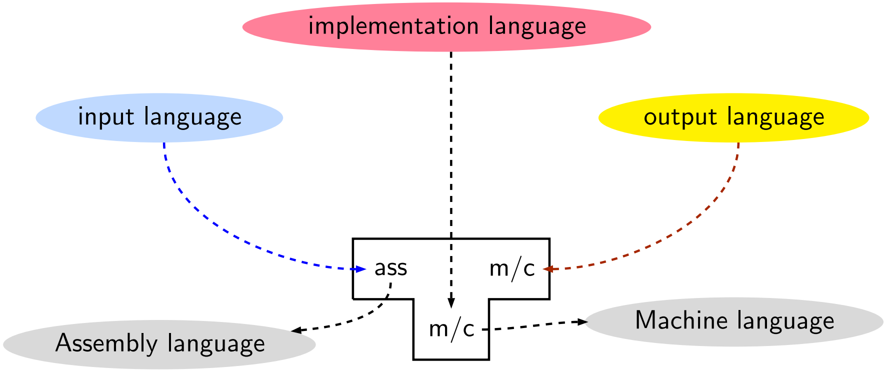
Explaining Bootstrapping: T Notation for a Compiler



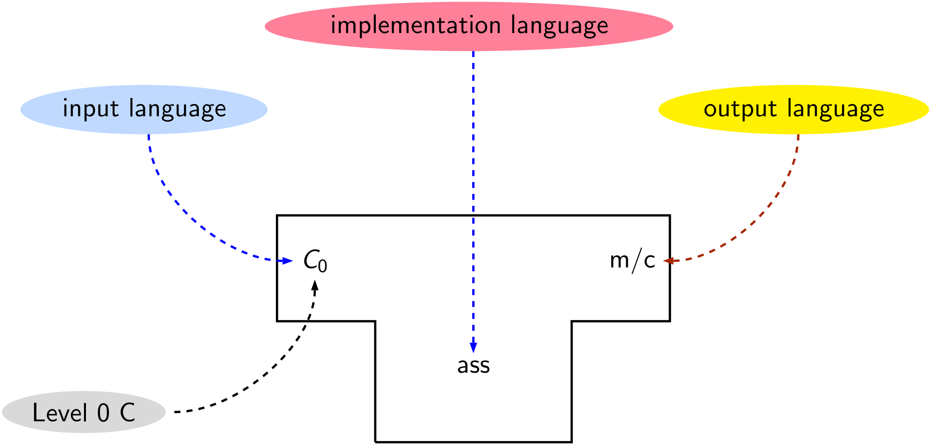
Bootstrapping: The Conventional View



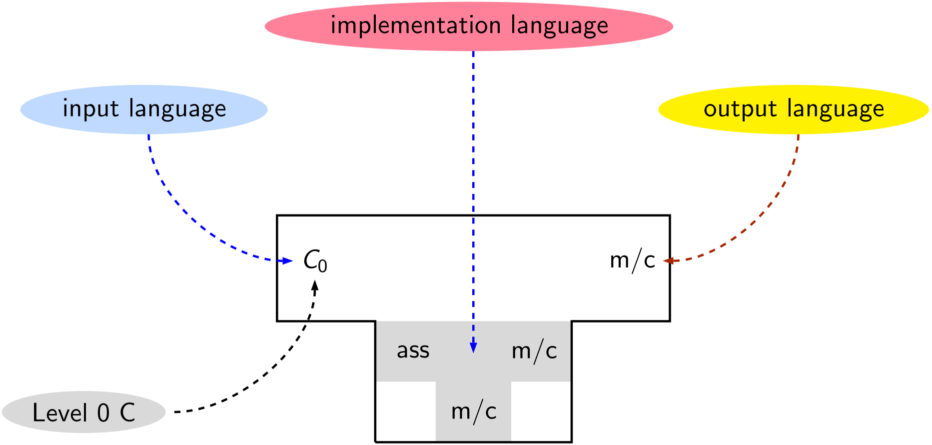
Bootstrapping: The Conventional View



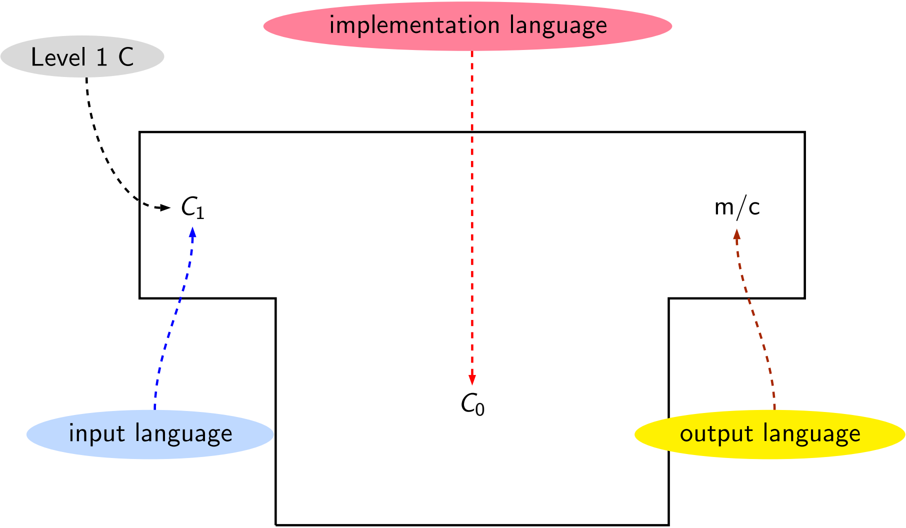
Bootstrapping: The Conventional View



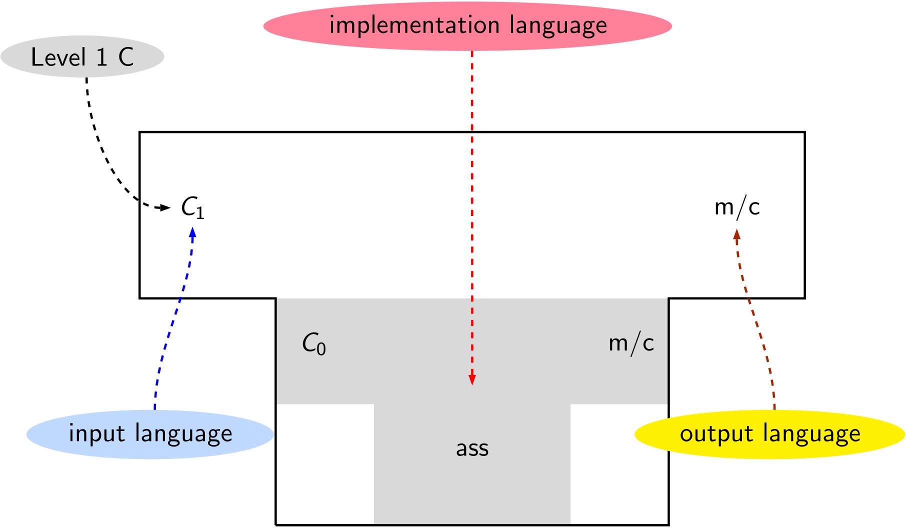
Bootstrapping: The Conventional View



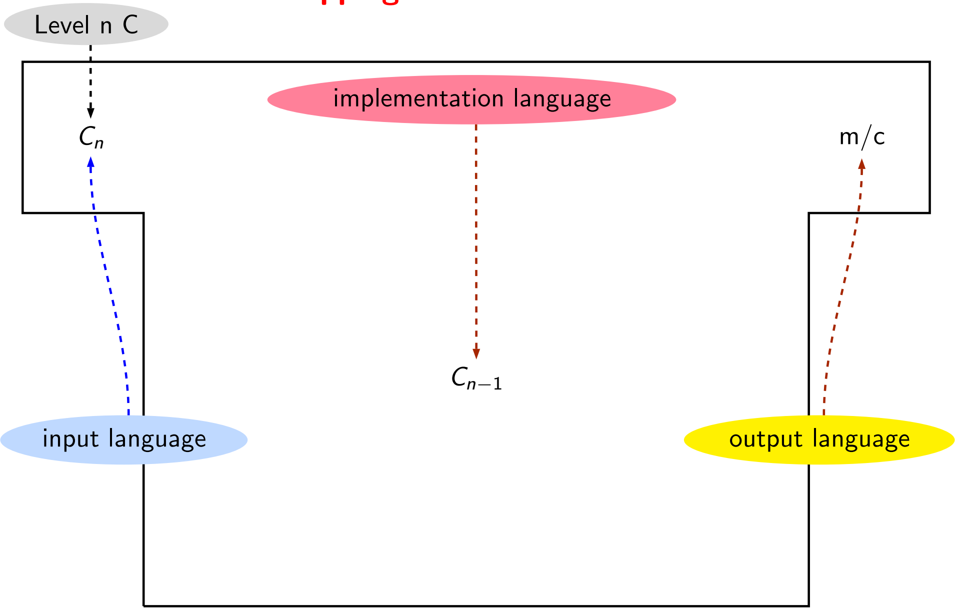
Bootstrapping: The Conventional View



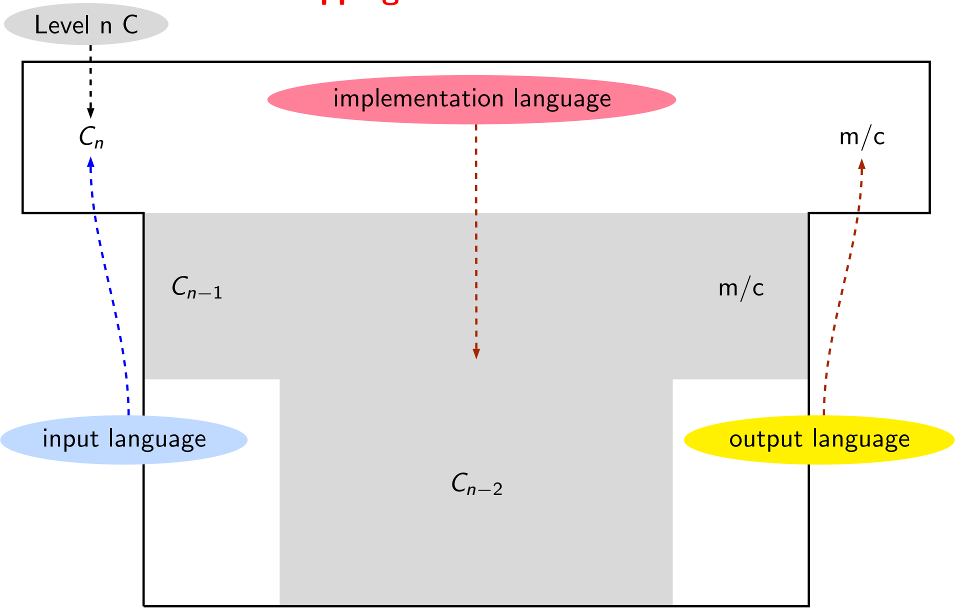
Bootstrapping: The Conventional View



Bootstrapping: The Conventional View



Bootstrapping: The Conventional View



Course Plan

cs320 Coverage and Pedagogy

- Coverage
  - Scanning, Parsing, Static Semantics, Runtime Support, Code Generation
  - Code Optimization, Register Allocation (may be omitted)
- Pedagogy
  - Lectures
  - Slides will be made available on moodle
  - Asynchronous discussions on moodle discussion forum
  - Tutorial problems on moodle
- Evaluation
  - Two quizzes, mid sem, end sem, and class participation

cs306 Coverage

- Coverage
  - Incremental construction of SCLP
  - Reference implementation with some test cases will be provided
- Pedagogy
  - Five assignments common to all students
    - Implementation of additional features may fetch bonus credit
    - Independent projects may be allowed to replace some of the assignments
  - Roughly two weeks per submission
  - Work do be done in a personalized VM in the lab machines or on your laptop
  - Groups of two
- Evaluation will be by running diff on the output
  - Standard file names and directory names must be used
  - It will not be possible to entertain violations
  - Use your creativity inside your code, not in file names, Makefile commands and program output

Evaluation Plan and Grading

- cs320

  Mid-Sem (30%) and End-Sem (40%)

  Quizzes (20%)

  Class Participation (10%)

  A viva may be factored in later

- cs306

  Five assignments to be done on coffre (90%)

  Class Participation (10%)

  A viva may be factored in later

- Additional opportunities for students to pass the course

Expectation Management

FAQ (1)

- Q. Can I form a group of 3 for cs306 assignments?

  A. The credit of each student will be proportionately reduced

- Q. Can I form a group of 1 for cs306 assignments?

  A. No, I do not want to increase the number of groups

- Q. Why can't I take my code out of coffre server even at the end of the course?

  Your take away from the course is your learning and not your code. Besides, we have had students publishing their code on public repositories which conflicts with the goals of the course for subsequent batches

FAQ (3)

- Q. I sent a message to you on MS Teams but did not get any response!

  A. I don't use MS Teams

- Q. I sent a message to you on Moodle but did not get any response!

  A. Post queries on moodle forums so they get tracked by multiple people

- Q. What are your office hours?

  A. In my experience, office hours have not been used by students in past. So instead of blocking my time with no takers, I prefer to have the freedom to schedule other activities/meetings

  However, if many students feel that they would like to use my time in office hours, I don't mind blocking my time

  Contact your CRs and I will coordinate with them

FAQ (4)

- Q. How do we communicate with you outside of the class?

  A. Talk me just after the class, minimize the email communication

  For issues that are likely to be of common interest, please write on moodle discussion forum

  For issues of personal interests rather than common interests, send an email to uday@cse.iitb.ac.in with a copy to nisha@cse.iitb.ac.in

  The subject header of every email sent to me must contain the word "cs320" or "cs306"; otherwise the mail may be mis-classified and may not be attended to for a long time

FAQ (5)

- Q. Is attendance compulsory?

  A. Yes, if you need my time and attention. No, otherwise.

  If your attendance is less than 80%, your emails, questions, queries, grievances about evaluation and marks, will be ignored totally. If you need to talk to me for *anything* you must attend the classes.

  Both attendance and participation is highly desirable. Participation can be in the form of asking questions in the class, discussions in meetings, on moodle forums, pointing out bugs in the reference implementation, responding to the posts made by others etc.

  In the absence of any such participation, I will have to use attendance as a proxy

- Q. Why do you insist on students attending and participating? Why can't we learn independently?

  A. Studying is like evaluating expressions that have lots of free variables. Regular participation ensures that these variables are bound to right values

  Familiarity with the notations and conventions used in the class simplifies understanding and evaluation and makes grades consistent and reliable

Things that I Discourage and Disapprove

- Asking questions only in the last two days before exams

  I will not provide clarifications two days before the exam
- Direct communication with the TAs about policies, evaluations, requests for extensions, or marks

  All communication must be with me. TAs are not authorized to respond to you directly on these matters without my explicit permission to do so
- Expecting instantaneous responses, making a request at the last moment

  Email responses will be provided in batch mode. And there may be no response to some emails. Please do not remind me
- Trying to push your luck by assuming that requests for concessions can only give benefit but no harm

  I will listen sympathetically, and may explain the rationale behind my decision but will not allow persuasions nor will try to convince you
- Feigning ignorance about the policies that have been described clearly in the class, slides, emails, or moodle announcements

  I have no sympathy for such students—sorry, bad luck!

Further Expectation Management

- Things that I like and strongly encourage
  - Asking questions, reporting bugs in reference implementation
  - Free, frank, and respectful communication
- Thing that I detest and despise
  - Cheating
  - Grade Litigation

Cheating

- Collaboration in learning or discussions about your code are fine
  But your answers in exam and your code in submit MUST BE YOURS
- No compromises on it
- Cheating is also a way of denying yourself an opportunity of learning
- Would you advice your younger siblings, nephews, nieces, and your children to copy?
- If you still want to compromise on your integrity, don't even think of doing so in cs320 and cs306

Pre-empting Cheating

- cs320 exams will be designed in a manner that attending the lectures would be sufficient to pass them
- Getting high marks in two quizzes and mid-sem will guarantee passing even if you get a zero in the end-sem
- Getting full marks in first two assignments in cs306 will guarantee passing even if you do not submit the remaining assignments
- Your lab work will be in coffre VM
  - Flexibility of doing work at your convenience
  - Can be used on multiple devices
  - You are free to access internet while you work on your programming
  - All programming and submissions within coffre
  - All submissions will undergo through plagiarism checking
- Additional opportunities will be provided to students to pass the courses

Hopefully, I will make it harder for you to fail

Grade Litigation

- Grade Litigation. Requests for accepting incomplete or inaccurate answers, requests to condone errors in answers or implementations, seeking waivers motivated by improving chances of grades

  Persistent demands in the disguise of requests with a sense of entitlement
- Grade litigation is easy to handle for small classes but not for large classes
- My take motivated by consistency and reliability of grades in large classes
  - Marks are given for rigorous demonstration of knowledge *and* the skill of demonstrating the knowledge
  - Most students rely on knowledge but ignore the skill part and end up writing vague or incomplete answers
  - You cannot claim marks for indirectly showing that you possess the knowledge to solve a question

    You need to demonstrate it directly by showing
    - the skill of using the right knowledge, and
    - the diligence of solving the questions using the notations and conventions used in the class

Grade Litigation

- Practically, grade litigation in large classes amounts to a Denial of Service Attack

  Takes the time and energy away from
  - The students who need and deserve my time and attention
  - Improving my lectures, explanation, and course material
  - Designing interesting exams
- I do not take this denial of service attack, kindly

Pre-empting The Grade Litigation for Large Classes: cs320

- By design, the answers will be objective and precise and not subjective
- Answers will be subdivided into parts and there will be no partial marks other than the subdivision by parts
- All answers will be published and corrections, if any, as well as possible valid variations, will be invited before finalizing the marks
- Corrections and acceptable variations in the answers will be published and no requests for consideration of any other answers will be entertained
- Evaluated answer sheets will be made available for you to scan them and go through them at leisure to ensure that the evaluation is consistent with the published answers
- A grievance form will be floated and valid grievances will be addressed (no discussions over email/phone, please)

Pre-empting The Grade Litigation for Large Classes: cs306

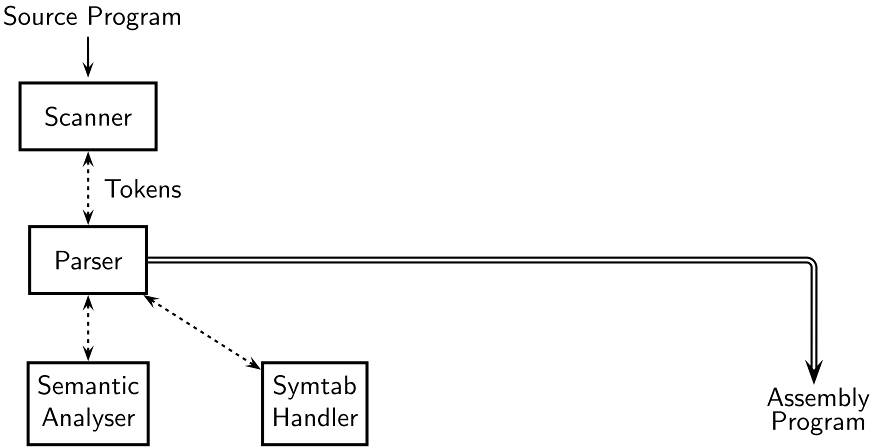
- A reference implementation of the compiler with test cases will be provided; you can run it on your own test cases to understand its behaviour
- You can submit the assignments late for reduced credit. Exact details are provided in moodle.
- You will have a total four late days of submission without late penalty through out the semester. You can use them for any assignment. Exact details are provided on moodle
- Full grammar and class hierarchies used in the reference implementation will be provided; you are free to use you own grammar and classes but the output must match the output of the reference implementation
- The scripts and the test cases used for evaluation will be published
- A grievance form will be floated and valid grievances will be addressed (no discussions over email/phone, please)

Are you ready for the fun?

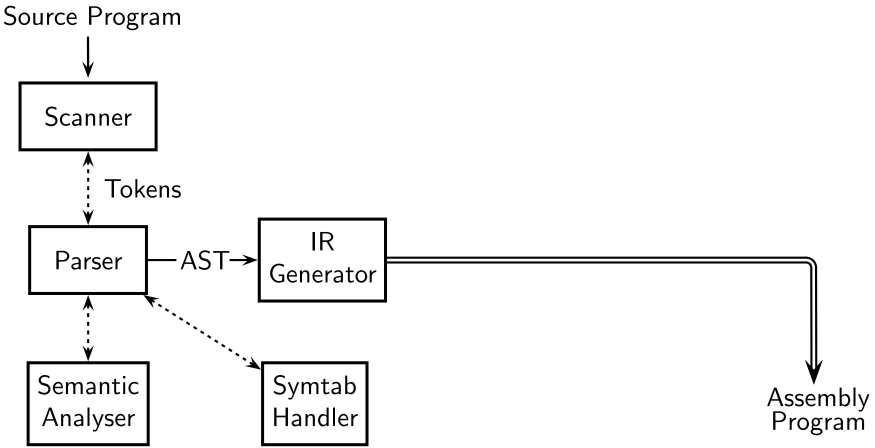
An Overview of Compilation Phases

The Structure of a Simple Compiler

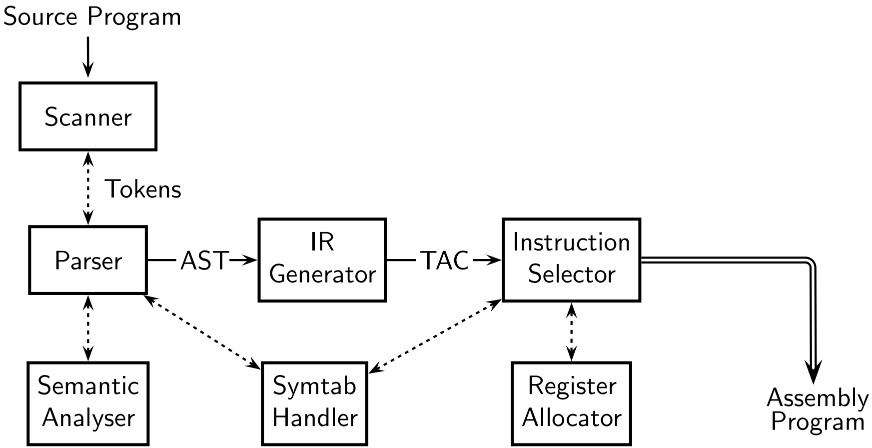
The Structure of a Simple Compiler



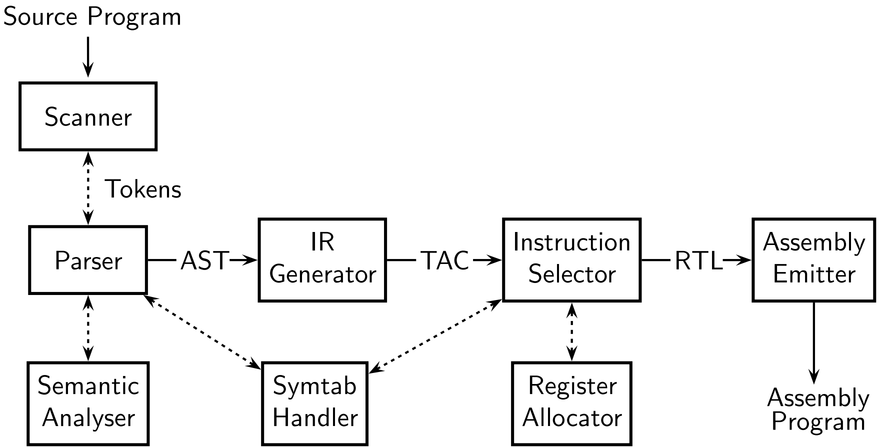
The Structure of a Simple Compiler



The Structure of a Simple Compiler



The Structure of a Simple Compiler



Translation Sequence in Our Compiler: Scanning and Parsing

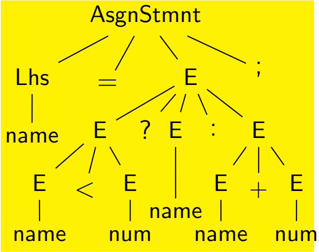
Input

```
a = b<10 ? b : c+5;
```

Translation Sequence in Our Compiler: Scanning and Parsing

Input

`a = b<10 ? b : c+5;`



Parse Tree

How the input is actually stored in the memory

`a␣=␣b␣<␣10␣?␣b␣:␣c␣+␣5␣;␣`

How we want to see it

`[a]␣[=]␣[b]␣[<]␣[10]␣[?]␣[b]␣[:]␣[c]␣[+]␣[5]␣[;]␣`

Issues:

- Grammar rules, terminals, non-terminals
- Order of application of grammar rules

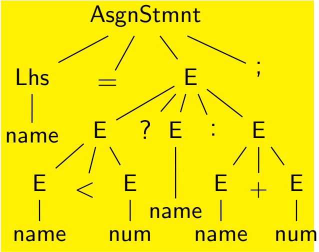
  eg. is it `(a = b<10?)` followed by `(b:c)`?
- Values of terminal symbols

  eg. string "10" vs. integer number 10.

Translation Sequence in Our Compiler: Semantic Analysis

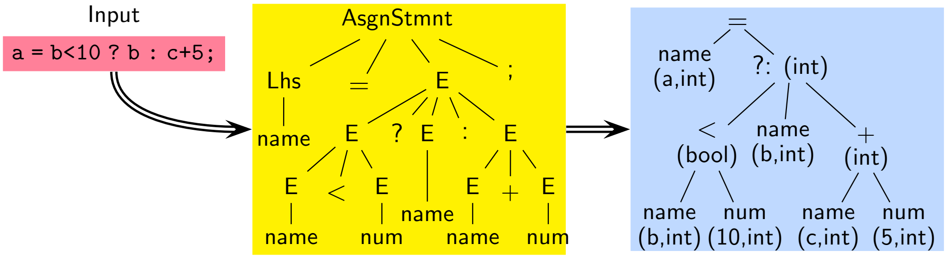
Input

`a = b<10 ? b : c+5;`



Parse Tree

Translation Sequence in Our Compiler: Semantic Analysis



Parse Tree

Abstract Syntax Tree
(with attributes)

Issues:

- Symbol tables

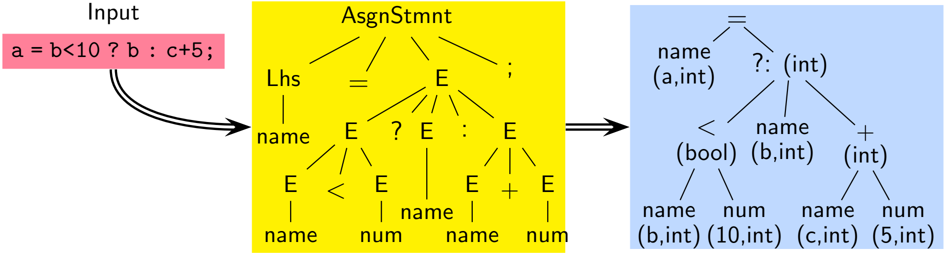
  Have variables been declared? What are their types? What is their scope?

- Type consistency of operators and operands

  The result of computing `b<10?` is bool and not int

Translation Sequence in Our Compiler: IR Generation

Input



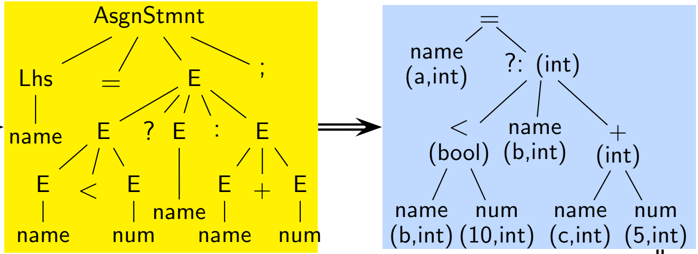
Parse Tree

Abstract Syntax Tree (with attributes)

Translation Sequence in Our Compiler: IR Generation

Input

`a = b<10 ? b : c+5;`



Parse Tree

Abstract Syntax Tree (with attributes)

Issues:

- Convert to three address code (TAC) separating data and control flow

  Simplifies optimization
- Linearise control flow by flattening nested control constructs

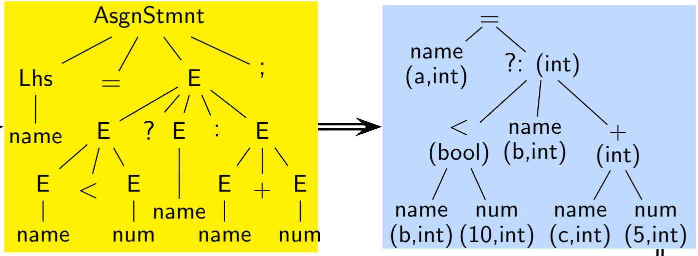
TAC List

$T_1 = b < 10$
$T_2 = \neg T_1$
if T_2 goto L0
$T_3 = b$
goto L1:
L0: $T_3 = c + 5$
L1: $a = T_3$

Translation Sequence in Our Compiler: Instruction Selection

Input

`a = b<10 ? b : c+5;`



Parse Tree

Abstract Syntax Tree (with attributes)

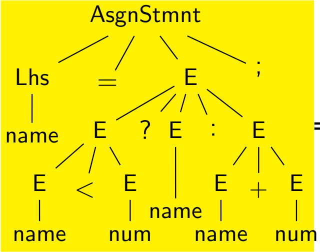
TAC List

$T_1 = b < 10$
$T_2 = \neg T_1$
if T_2 goto L0
$T_3 = b$
goto L1:
L0: $T_3 = c + 5$
L1: $a = T_3$

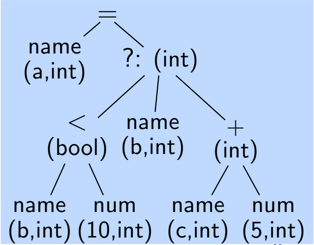
Translation Sequence in Our Compiler: Instruction Selection

Input

`a = b<10 ? b : c+5;`



Parse Tree



Abstract Syntax Tree
(with attributes)

TAC List

$T_1 = b < 10$
$T_2 = \neg T_1$
if T_2 goto L0
$T_3 = b$
goto L1:
L0: $T_3 = c + 5$
L1: $a = T_3$

RTL List

load $v0 \leftarrow b$
slti $t1 \leftarrow v0, 10$
not $t2 \leftarrow t1$
if $t2$ goto L0
load $t3 \leftarrow b$
goto L1
L0: load $t4 \leftarrow c$
addi $t3 \leftarrow t4, 5$
L1: store $a \leftarrow t3$

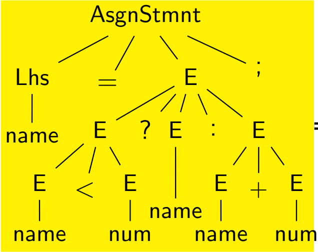
Issues:

- Generate as few instructions as possible (list shown here is unoptimized)
- Use temporaries and local registers

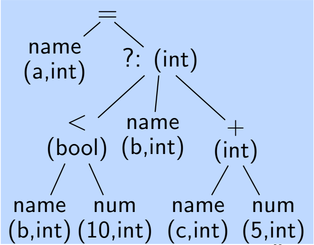
Translation Sequence in Our Compiler: Emitting Instructions

Input

`a = b<10 ? b : c+5;`



Parse Tree



Abstract Syntax Tree
(with attributes)

TAC List

$T_1 = b < 10$
$T_2 = \neg T_1$
if T_2 goto L0
$T_3 = b$
goto L1:
L0: $T_3 = c + 5$
L1: $a = T_3$

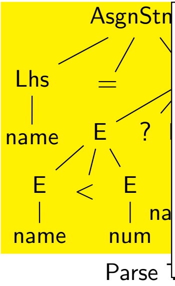
RTL List

load $v0 \leftarrow b$
slti $t1 \leftarrow v0, 10$
not $t2 \leftarrow t1$
if $t2$ goto L0
load $t3 \leftarrow b$
goto L1
L0: load $t4 \leftarrow c$
addi $t3 \leftarrow t4, 5$
L1: store $a \leftarrow t3$

Translation Sequence in Our Compiler: Emitting Instructions

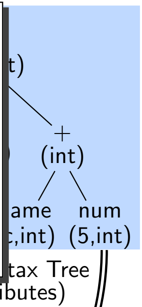
Input

`a = b<10 ? b : c+5;`



Issues:

- Offsets of variables in the stack frame
- Actual register numbers and assembly mnemonics
- Code to construct and discard activation records



TAC List

$$
\begin{array}{ll}
 & T_1 = b < 10 \\
 & T_2 = \neg T_1 \\
 & \text{if } T_2 \text{ goto L0} \\
 & T_3 = b \\
 & \text{goto L1:} \\
\text{L0:} & T_3 = c + 5 \\
\text{L1:} & a = T_3
\end{array}
$$

RTL List

$$
\begin{array}{ll}
 & \text{load } v0 \leftarrow b \\
 & \text{slti } t1 \leftarrow v0, 10 \\
 & \text{not } t2 \leftarrow t1 \\
 & \text{if } t2 \text{ goto L0} \\
 & \text{load } t3 \leftarrow b \\
 & \text{goto L1} \\
\text{L0:} & \text{load } t4 \leftarrow c \\
 & \text{addi } t3 \leftarrow t4, 5 \\
\text{L1:} & \text{store } a \leftarrow t3
\end{array}
$$

Assembly Code

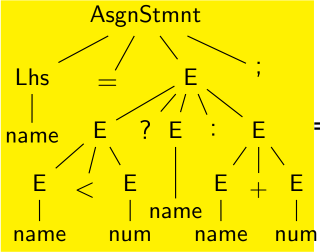
```
    lw   $v0, 4($fp)
    slti $t1, $t0, 10
    xori $t2, $t1, 1
    bgtz $t2, L0
    lw   $t3, 4($fp)
    b    L1
L0: lw   $t4, 8($fp)
    addi $t3, $t4, 5
L1: sw   0($fp), $t3
```

Translation Sequence in Our Compiler: Emitting Instructions

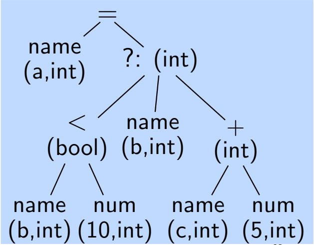
Input

```
a = b<10 ? b : c+5;
```

Output



Parse Tree



Abstract Syntax Tree
(with attributes)

TAC List

$T_1 = b < 10$
$T_2 = \neg T_1$
if T_2 goto L0
$T_3 = b$
goto L1:
L0: $T_3 = c + 5$
L1: $a = T_3$

RTL List

load $v0 \leftarrow b$
slti $t1 \leftarrow v0, 10$
not $t2 \leftarrow t1$
if $t2$ goto L0
load $t3 \leftarrow b$
goto L1
L0: load $t4 \leftarrow c$
addi $t3 \leftarrow t4, 5$
L1: store $a \leftarrow t3$

Assembly Code

```
    lw   $v0, 4($fp)
    slti $t1, $t0, 10
    xori $t2, $t1, 1
    bgtz $t2, L0
    lw   $t3, 4($fp)
    b    L1
L0: lw   $t4, 8($fp)
    addi $t3, $t4, 5
L1: sw   0($fp), $t3
```

Observations

- A compiler bridges the gap between source program and target program

Observations

- A compiler bridges the gap between source program and target program
- Compilation involves gradual lowering of levels of the IR of an input program

Observations

- A compiler bridges the gap between source program and target program
- Compilation involves gradual lowering of levels of the IR of an input program
- The design of IRs is the most critical part of a compiler design
  - How many IRs should we have?
  - What are the details that each IR captures?

Observations

- A compiler bridges the gap between source program and target program
- Compilation involves gradual lowering of levels of the IR of an input program
- The design of IRs is the most critical part of a compiler design
    - How many IRs should we have?
    - What are the details that each IR captures?
- Practical compilers are desired to be retargetable

  $\Rightarrow$ Back ends should be generated from specifications

Why Is Compiler Construction a Relevant Course?

Even though very few people write compilers ...

Why Is Compiler Construction a Relevant Course?

Even though very few people write compilers . . .

- Translation and interpretation are fundamental CS at a conceptual level
  - Stepwise refinement Vs. look up
  - Analytics Vs. Transactional software

Why Is Compiler Construction a Relevant Course?

Even though very few people write compilers ...

- Translation and interpretation are fundamental CS at a conceptual level
  - Stepwise refinement Vs. look up
  - Analytics Vs. Transactional software
- Computer Science is all about building layers of abstractions and bridging the gaps between successive layers

Why Is Compiler Construction a Relevant Course?

Even though very few people write compilers . . .

- Translation and interpretation are fundamental CS at a conceptual level
  - Stepwise refinement Vs. look up
  - Analytics Vs. Transactional software
- Computer Science is all about building layers of abstractions and bridging the gaps between successive layers
- Knowing compilers internals makes a person a much better programmer

  Writing programs whose data is programs

Why Is Compiler Construction a Relevant Course?

Even though very few people write compilers . . .

- Translation and interpretation are fundamental CS at a conceptual level
  - Stepwise refinement Vs. look up
  - Analytics Vs. Transactional software
- Computer Science is all about building layers of abstractions and bridging the gaps between successive layers
- Knowing compilers internals makes a person a much better programmer

  Writing programs whose data is programs
- The beauty and enormity of compiling lies in
  - Raising the level of abstraction and bridging the gap without performance penalties
  - Meeting the expectations of users with a wide variety of needs

Where Can I Use the Lessons Learnt in Compiler Design?

- Compilers for all languages exist, so what can I do with the technology?

Where Can I Use the Lessons Learnt in Compiler Design?

- Compilers for all languages exist, so what can I do with the technology?
- Compiler techniques and tools have many applications
  - Parsers for HTML in web browser
  - Interpreters for javascript/flash
  - Machine code generation for high level languages
  - Software testing
  - Program optimization
  - Detection of malicious code
  - Design of new computer architectures
    Hardware-software codesign!
  - Hardware synthesis: VHDL to RTL translation
  - Compiled simulation to simulate designs written in VHDL

Credits: Adapted from the slides of Prof. Y. N. Srikant for NPTEL course on compilers

The Beauty and Enormity of Compiling

- Bridging the rather large gap between high and low level languages
  - Creating several layers of abstractions with smaller gaps
  - A great example of divide and conquer or stepwise refinement
- Developing and maintaining a rather large code base of millions of lines

The Beauty and Enormity of Compiling

- Bridging the rather large gap between high and low level languages
  - Creating several layers of abstractions with smaller gaps
  - A great example of divide and conquer or stepwise refinement
- Developing and maintaining a rather large code base of millions of lines
- Writing programs that read programs and write programs maintaining the semantics

The Beauty and Enormity of Compiling

- Bridging the rather large gap between high and low level languages
  - Creating several layers of abstractions with smaller gaps
  - A great example of divide and conquer or stepwise refinement
- Developing and maintaining a rather large code base of millions of lines
- Writing programs that read programs and write programs maintaining the semantics
- Extensive use of tools to generate modules from declarative specifications

  "Higher" level than HLLs

The Beauty and Enormity of Compiling

- Bridging the rather large gap between high and low level languages
  - Creating several layers of abstractions with smaller gaps
  - A great example of divide and conquer or stepwise refinement
- Developing and maintaining a rather large code base of millions of lines
- Writing programs that read programs and write programs maintaining the semantics
- Extensive use of tools to generate modules from declarative specifications
  "Higher" level than HLLs
- Handling every possible programs from an infinite set of possible programs

The Beauty and Enormity of Compiling

- Bridging the rather large gap between high and low level languages
  - Creating several layers of abstractions with smaller gaps
  - A great example of divide and conquer or stepwise refinement
- Developing and maintaining a rather large code base of millions of lines
- Writing programs that read programs and write programs maintaining the semantics
- Extensive use of tools to generate modules from declarative specifications
  "Higher" level than HLLs
- Handling every possible programs from an infinite set of possible programs
- Exploiting advanced features of rich computer architectures

The Beauty and Enormity of Compiling

- Bridging the rather large gap between high and low level languages
    - Creating several layers of abstractions with smaller gaps
    - A great example of divide and conquer or stepwise refinement
- Developing and maintaining a rather large code base of millions of lines
- Writing programs that read programs and write programs maintaining the semantics
- Extensive use of tools to generate modules from declarative specifications

  "Higher" level than HLLs
- Handling every possible programs from an infinite set of possible programs
- Exploiting advanced features of rich computer architectures
- Spanning both theory and practice (and everything in between) rather deeply

  Translating deep theory into general, efficient, and scalable, practice!

Modern Compilers Span Both Theory and Practice Deeply

Compiler design and implementation translates deep theory into general, efficient, and scalable, practice!

- Uses principles and techniques from many areas in Computer Science
  - The design and implementation of a compiler is a great application of software engineering
  - Makes practical application of deep theory and algorithms and rich data structures
  - Uses rich features of computer architecture

Translating Deep Theory into Affordable Practice

- Theory and algorithms
  - Mathematical logic: type inference and checking
  - Lattice theory: static analysis
  - Linear algebra: dependence analysis and loop parallelization
  - Probability theory: hot path optimization
  - Greedy algorithms: register allocation
  - Heuristic search: instruction scheduling
  - Graph algorithms: register allocation
  - Dynamic programming: instruction selection
  - Optimization techniques: instruction scheduling
  - Finite automata: lexical analysis
  - Pushdown automata: parsing
  - Fixed point algorithms: data-flow analysis

Credits: Adapted from the slides of Prof. Y. N. Srikant, IISc Bangalore

Translating Deep Theory into Affordable Practice

- Data structures
  - Sparse representations: scanner and parser tables
  - Stacks, lists, and arrays: Symbols tables
  - Trees: abstract syntax trees, expression trees
  - Graphs: control flow graphs, call graphs, data dependence graphs,
  - DAGs: Expression DAG
  - Representing machine details such as instruction sets, registers, etc.

Compilation Models

Compilation Models

Aho Ullman Model

Davidson Fraser Model

Compilation Models

Aho Ullman Model

Davidson Fraser Model

Front End ← Input Source Program

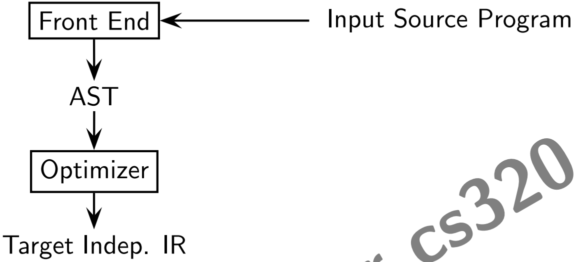
↓

AST

Compilation Models

Aho Ullman Model

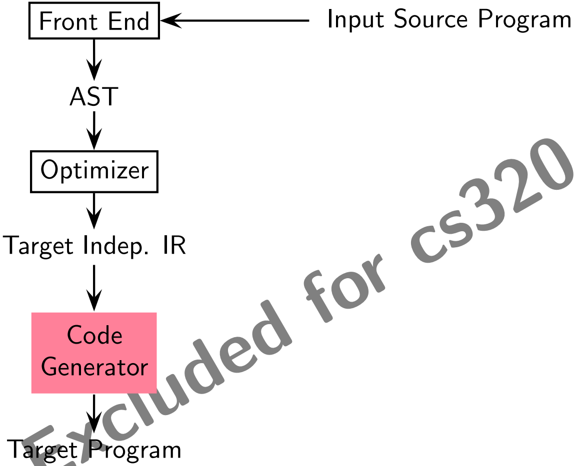
Davidson Fraser Model



Compilation Models

Aho Ullman Model

Davidson Fraser Model



Compilation Models

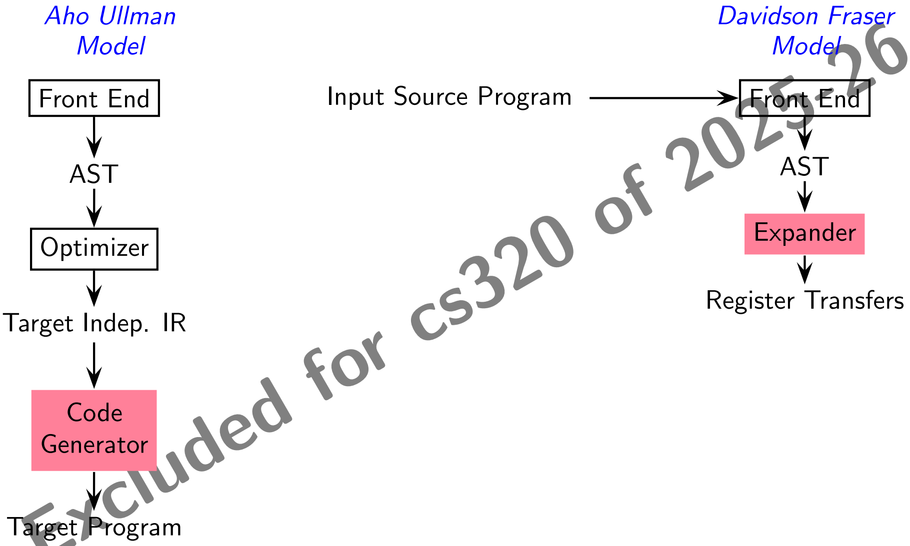
Aho Ullman Model

Front End → AST → Optimizer → Target Indep. IR → Code Generator → Target Program

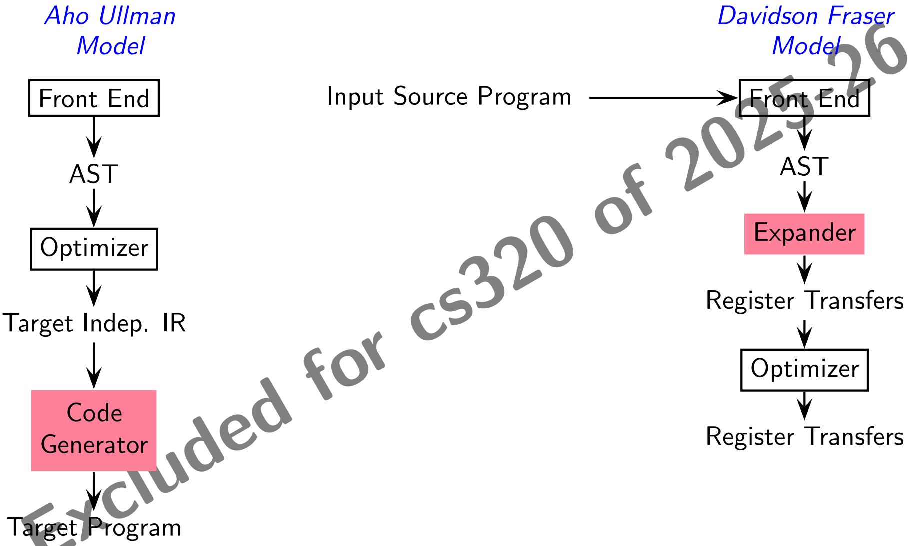
Davidson Fraser Model

Input Source Program → Front End → AST

Compilation Models



Compilation Models



Compilation Models

Aho Ullman Model

Front End → AST → Optimizer → Target Indep. IR → Code Generator → Target Program

Davidson Fraser Model

Input Source Program → Front End → AST → Expander → Register Transfers → Optimizer → Register Transfers → Recognizer → Target Program

Compilation Models

Aho Ullman Model

Front End → AST → Optimizer → Target Indep. IR → Code Generator → Target Program

Aho Ullman: Instruction selection

- over optimized IR using
- cost based tree tiling matching

Davidson Fraser: Instruction selection

- over AST using
- simple full tree matching based algorithms that generate
- naive code which is
  - target dependent, and is
  - optimized subsequently

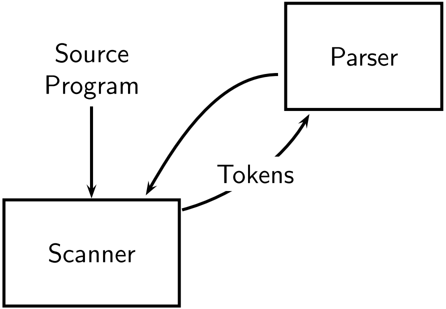
Davidson Fraser Model

Front End → AST → Expander → Register Transfers → Optimizer → Register Transfers → Recognizer → Target Program

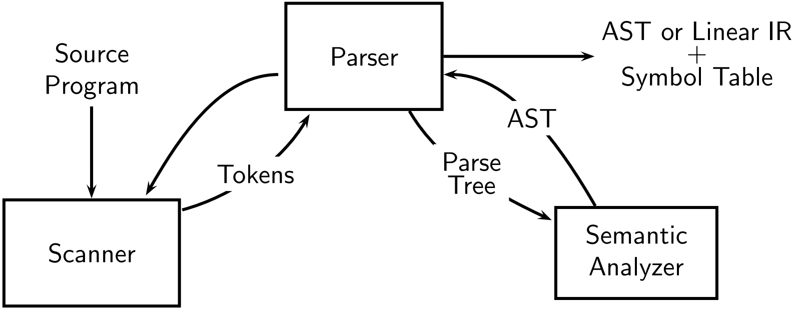
Typical Front Ends

$$\boxed{\text{Parser}}$$

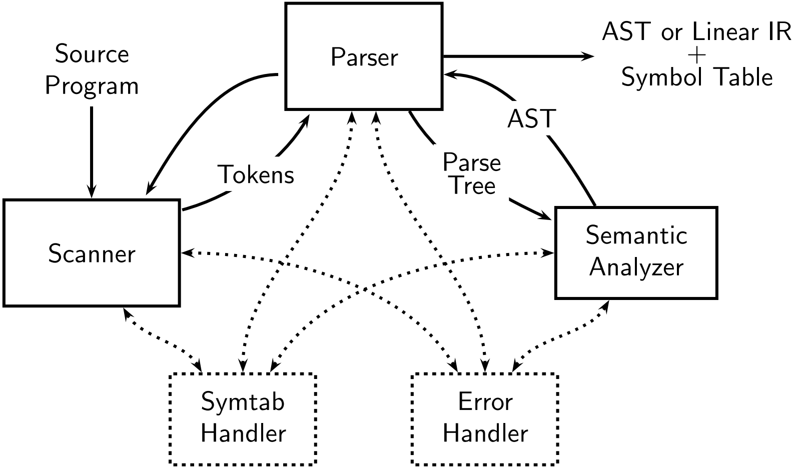
Typical Front Ends



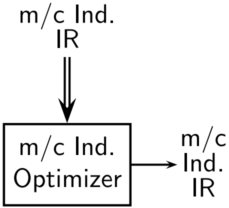
Typical Front Ends



Typical Front Ends



Typical Back Ends



- Compile time evaluations
- Eliminating redundant computations

Typical Back Ends

m/c Ind. IR ⇓ **m/c Ind. Optimizer** → m/c Ind. IR → **Code Generator** → m/c Dep. IR

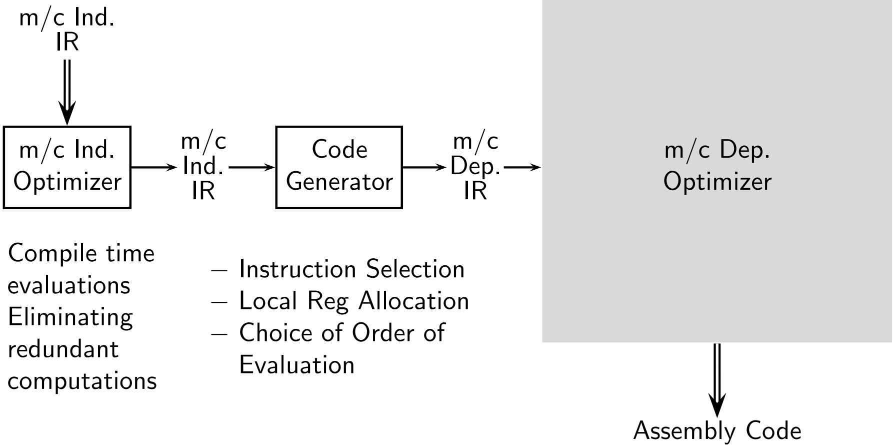
m/c Ind. Optimizer:

– Compile time evaluations
– Eliminating redundant computations

Code Generator:

– Instruction Selection
– Local Reg Allocation
– Choice of Order of Evaluation

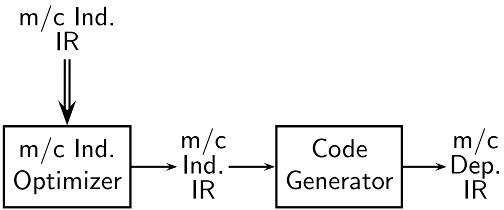
Typical Back Ends



- Compile time evaluations
- Eliminating redundant computations

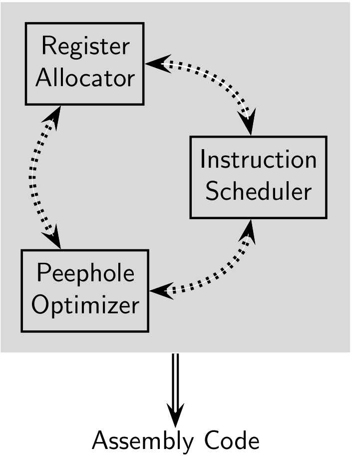
- Instruction Selection
- Local Reg Allocation
- Choice of Order of Evaluation

Typical Back Ends

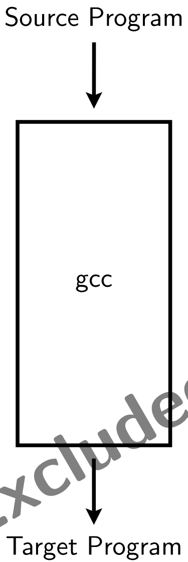


- Compile time evaluations
- Eliminating redundant computations

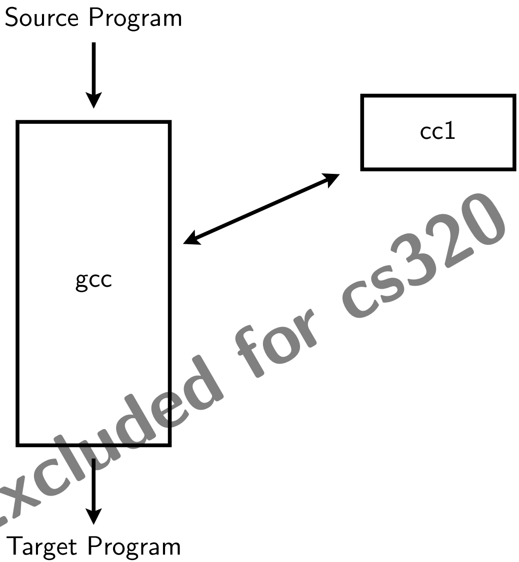
- Instruction Selection
- Local Reg Allocation
- Choice of Order of Evaluation



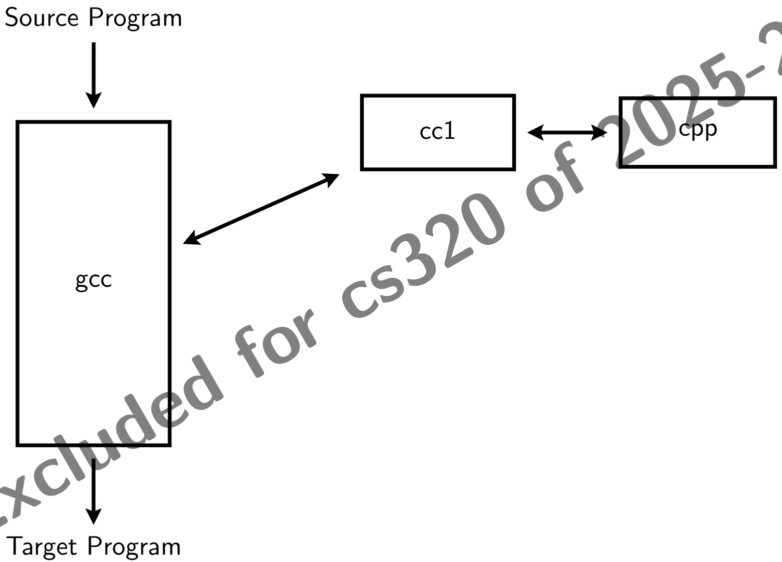
The GNU Tool Chain for C



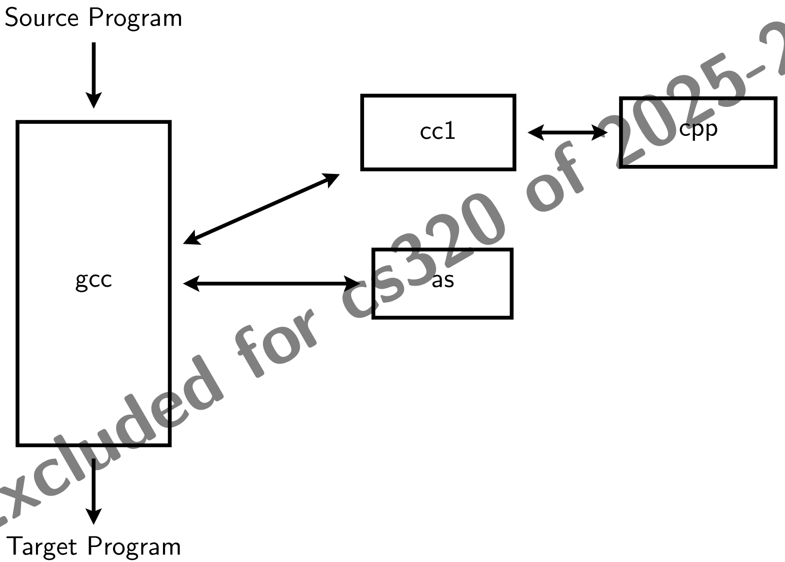
The GNU Tool Chain for C



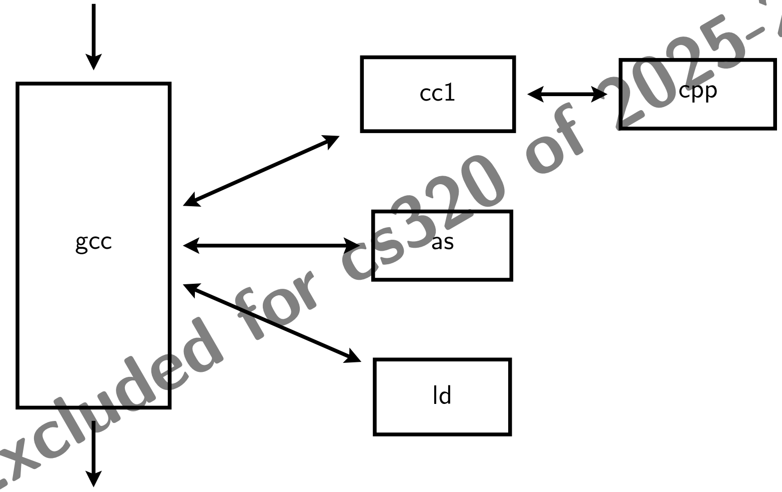
The GNU Tool Chain for C



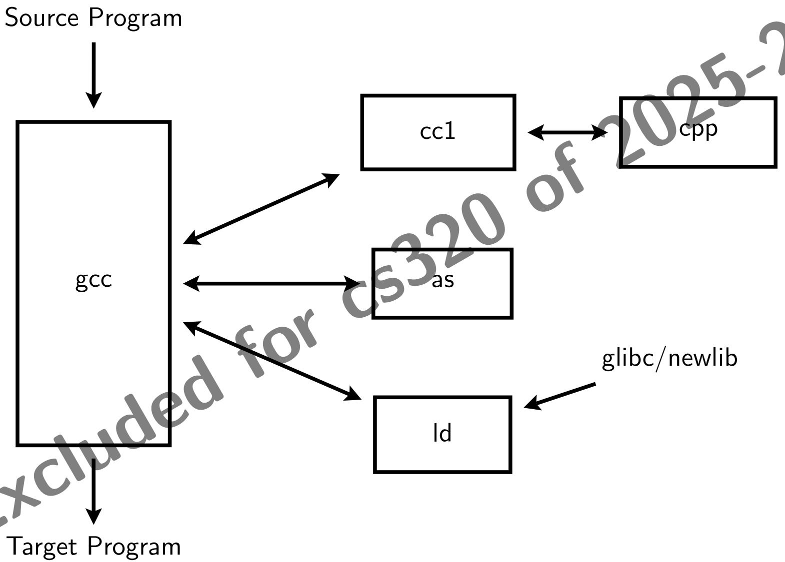
The GNU Tool Chain for C



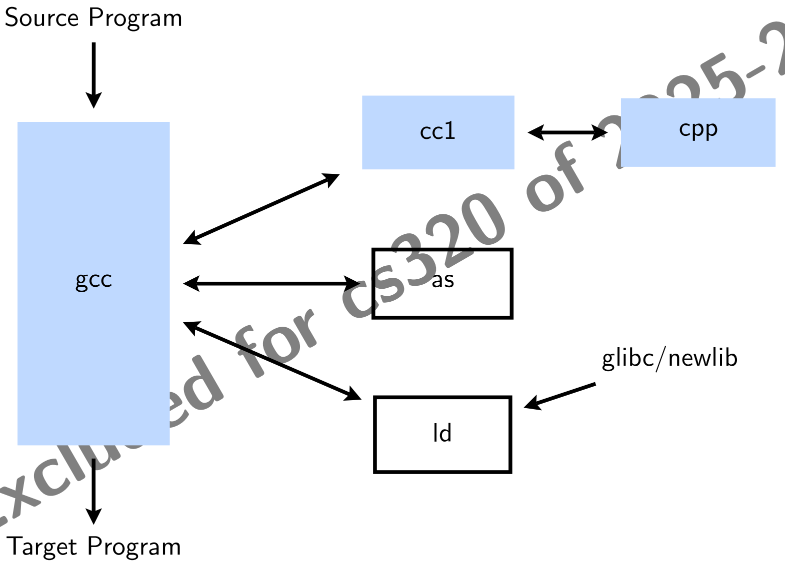
The GNU Tool Chain for C



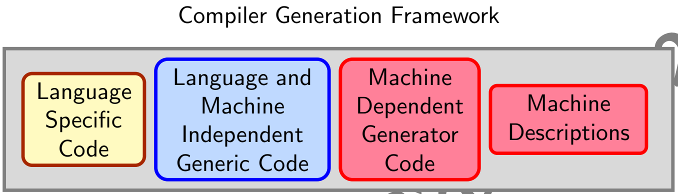
The GNU Tool Chain for C



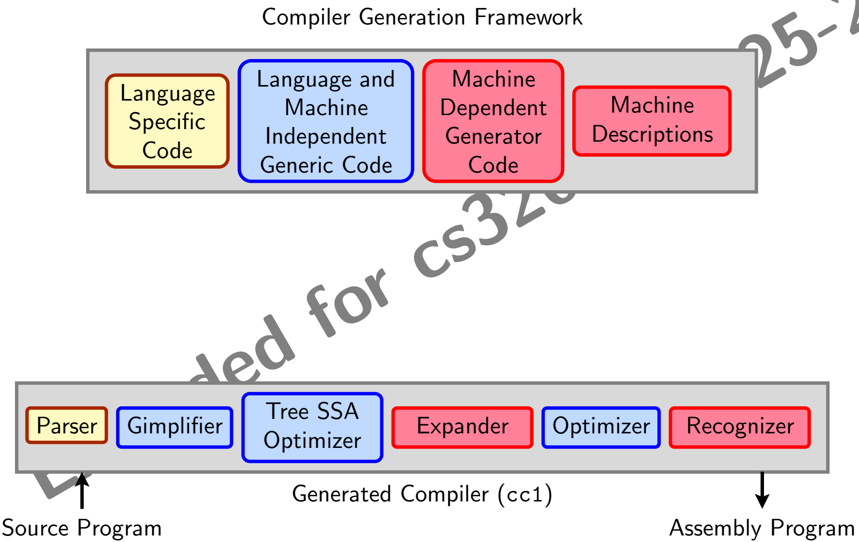
The GNU Tool Chain for C



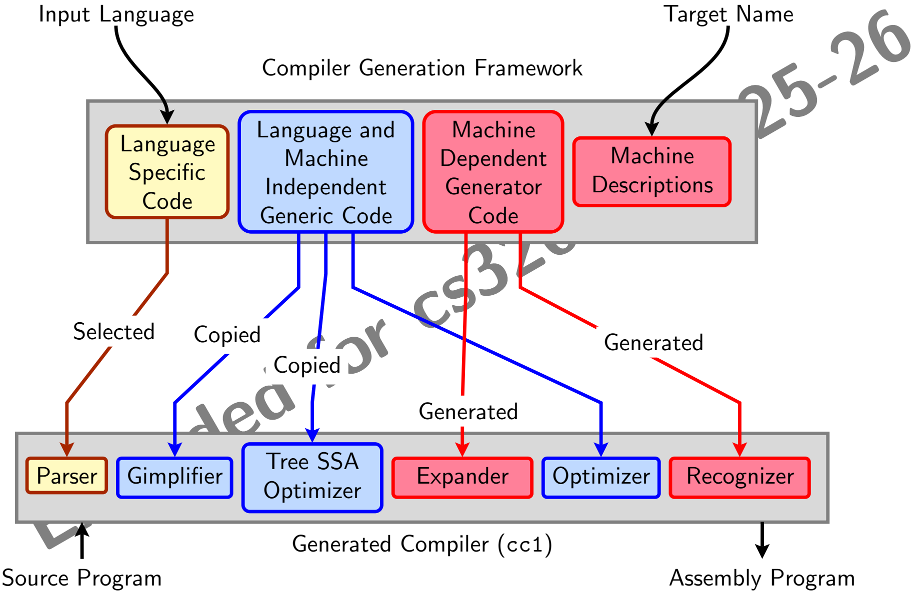
The Architecture of GCC



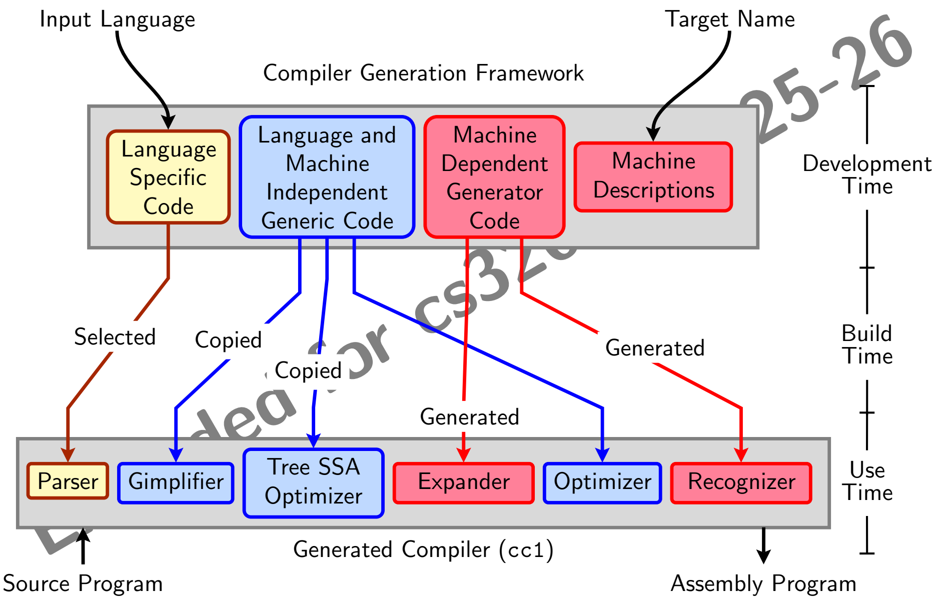
The Architecture of GCC



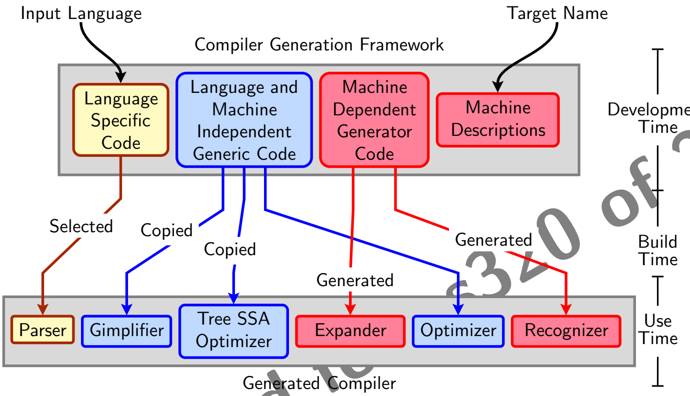
The Architecture of GCC



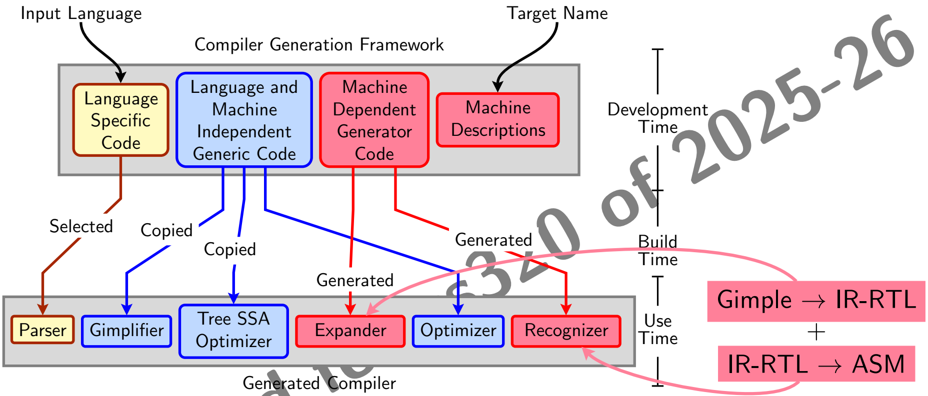
The Architecture of GCC



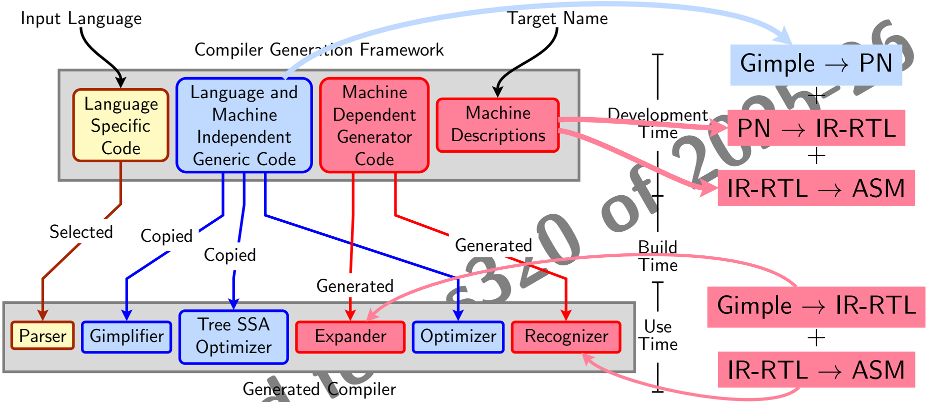
GCC Retargetability Mechanism



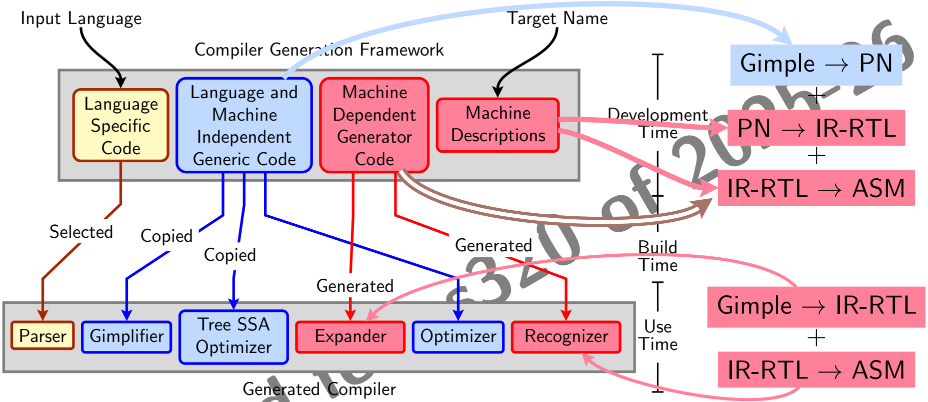
GCC Retargetability Mechanism



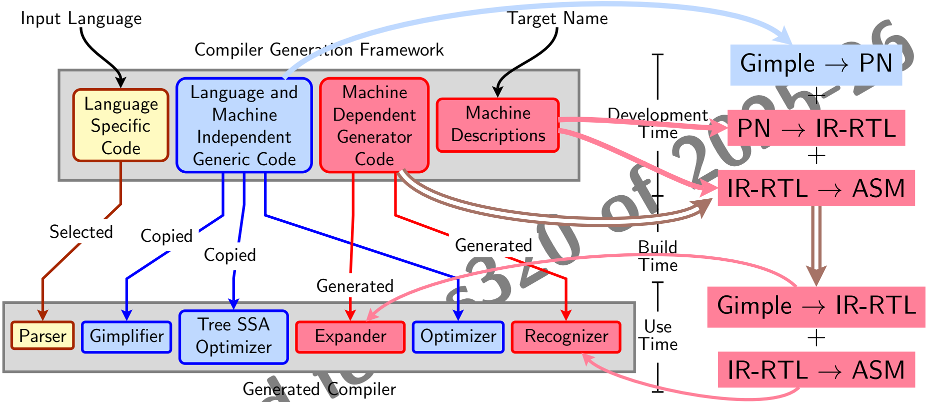
GCC Retargetability Mechanism



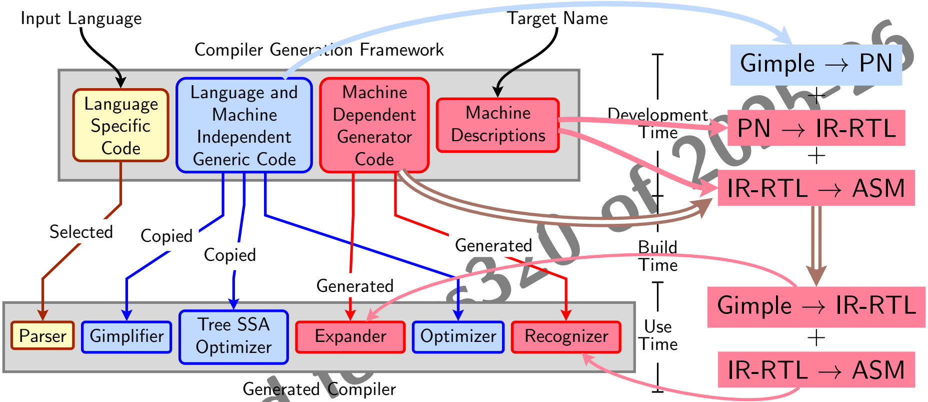
GCC Retargetability Mechanism



GCC Retargetability Mechanism



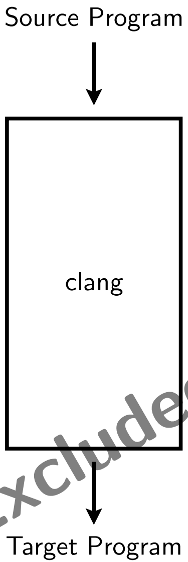
GCC Retargetability Mechanism



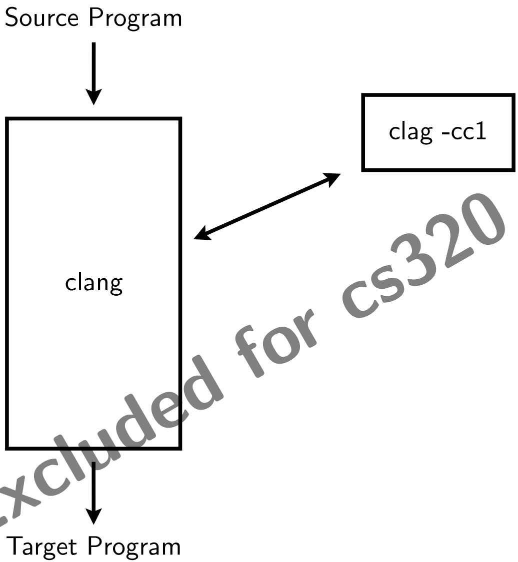
The generated compiler uses an adaptation of the Davidson Fraser model

- Generic expander and recognizer
- Machine specific information is isolated in data structures
- Generating a compiler involves generating these data structures

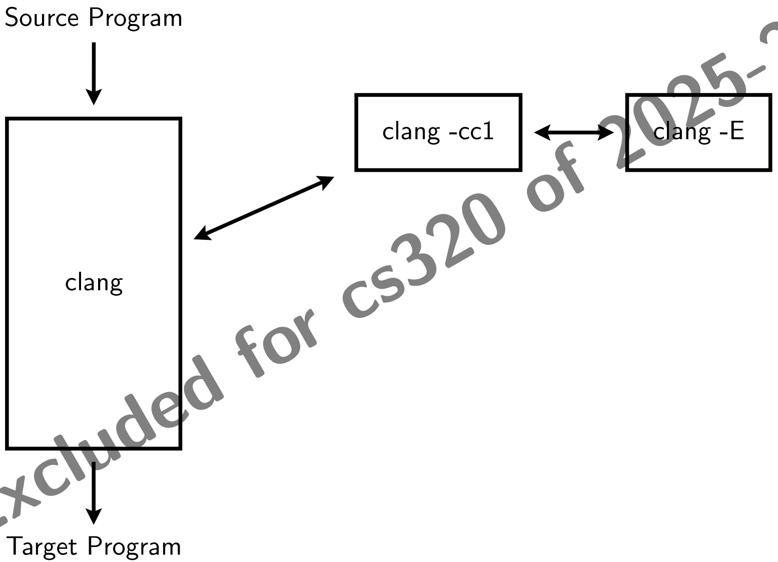
The LLVM Tool Chain for C



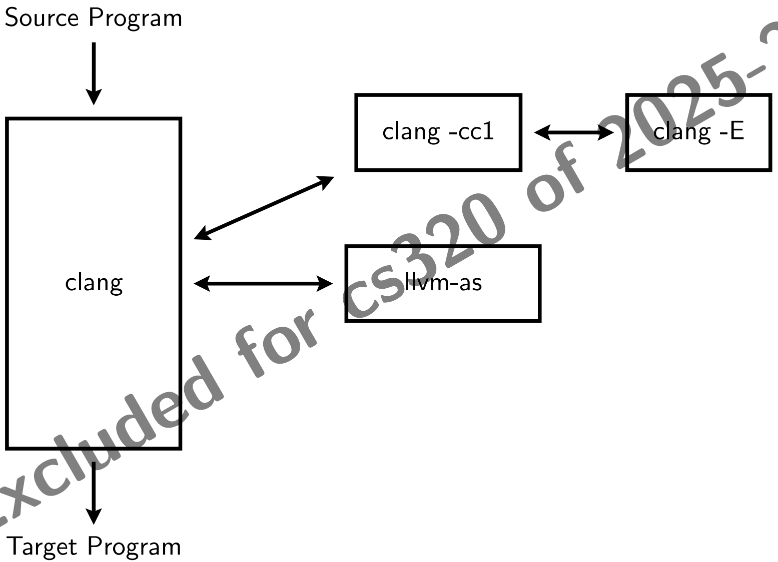
The LLVM Tool Chain for C



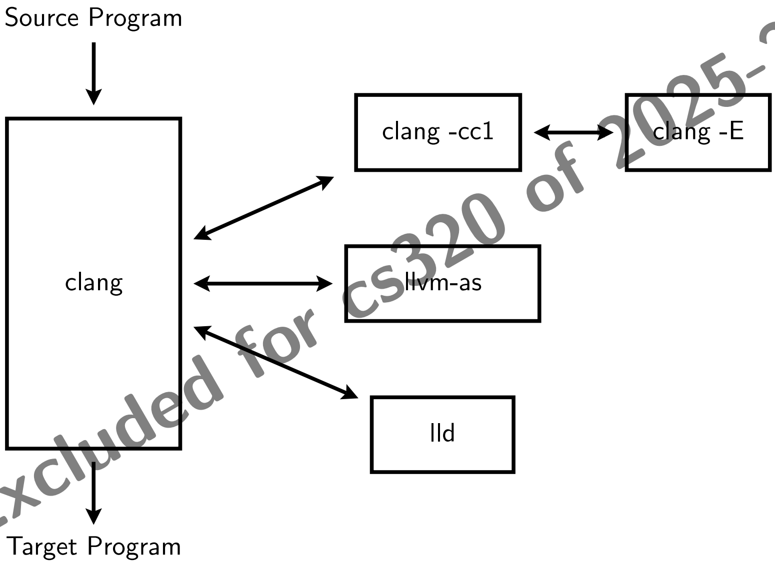
The LLVM Tool Chain for C



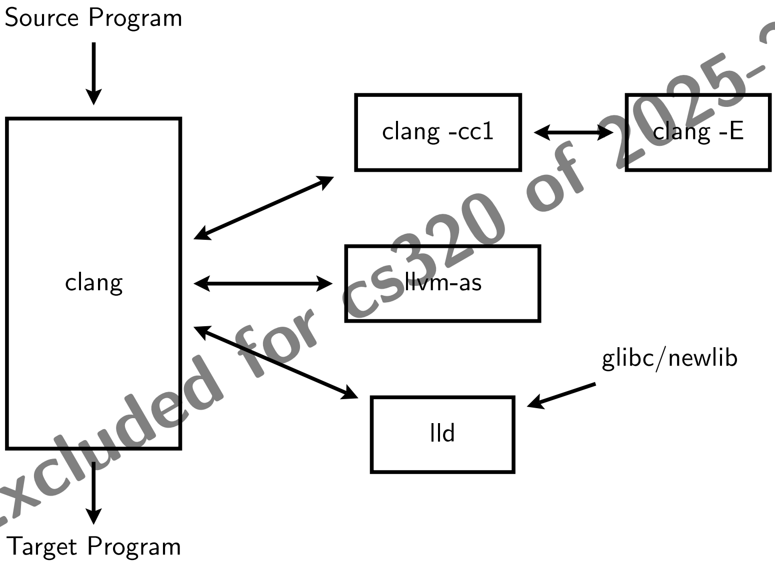
The LLVM Tool Chain for C



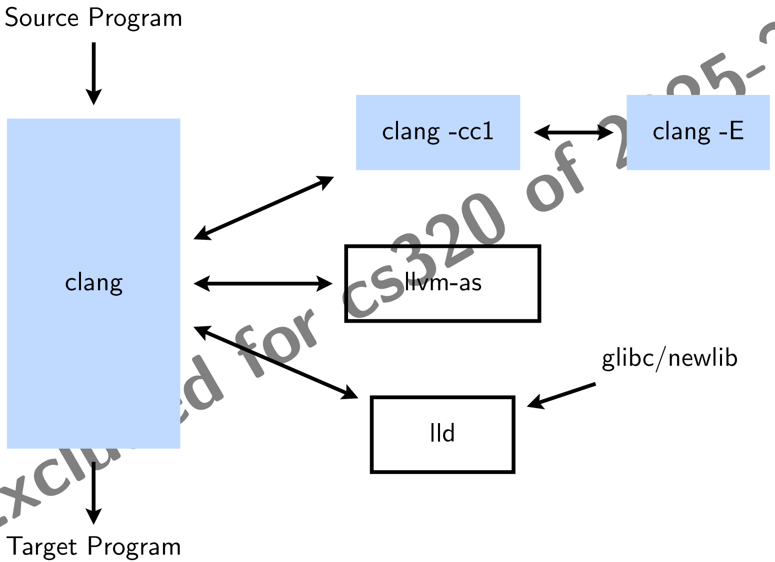
The LLVM Tool Chain for C



The LLVM Tool Chain for C

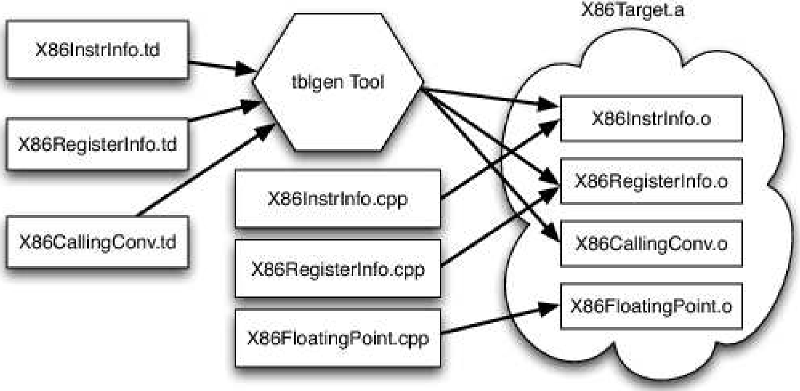


The LLVM Tool Chain for C



LLVM Retargetability Mechanism

Simplified x86 Target Definition



Reproduced from https://www.aosabook.org/en/llvm.html

Building a Compiler: Terminology

- The sources of a compiler are compiled (i.e. built) on *Build system*, denoted BS.
- The built compiler runs on the *Host system*, denoted HS.
- The compiler compiles code for the *Target system*, denoted TS.

The built compiler itself runs on HS and generates executables that run on TS.

Variants of Compiler Builds

BS = HS = TS	Native Build
BS = HS ≠ TS	Cross Build
BS ≠ HS ≠ TS	Canadian Cross

Example

Native i386: built on i386, hosted on i386, produces i386 code.
Sparc cross on i386: built on i386, hosted on i386, produces Sparc code.

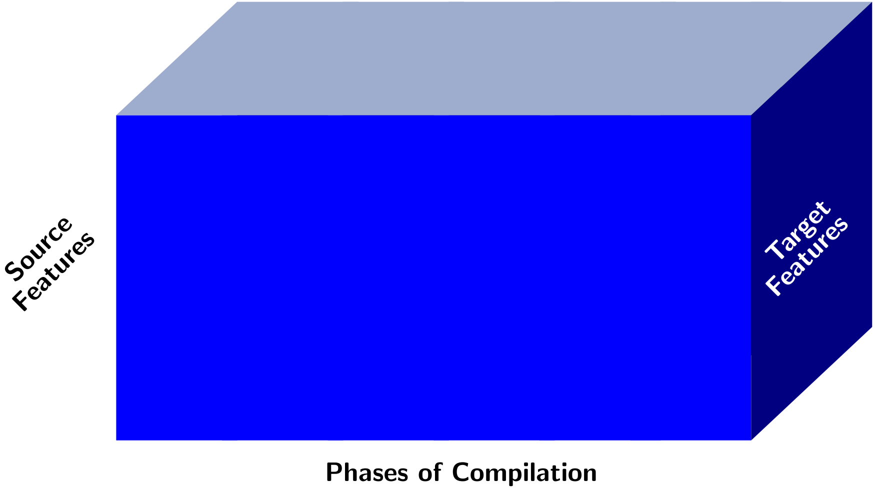
Bootstrapping: GCC View

- Language need not change, but the compiler may change
  Compiler is improved, bugs are fixed and newer versions are released
- To build a new version of a compiler given a built old version:
  - Stage 1: Build the new compiler using the old compiler
  - Stage 2: Build another new compiler using compiler from stage 1
  - Stage 3: Build another new compiler using compiler from stage 2
    Stage 2 and stage 3 builds must result in identical compilers

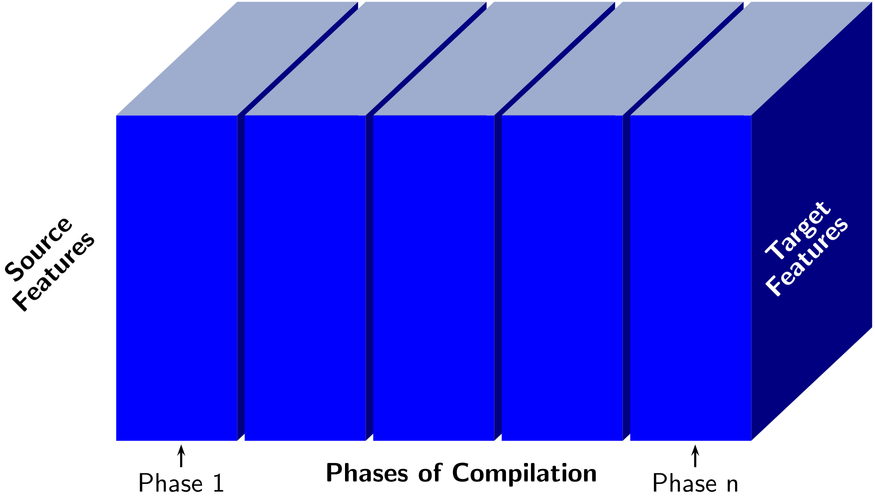
⇒ Building cross compilers stops after Stage 1!

Incremental Construction of Compilers

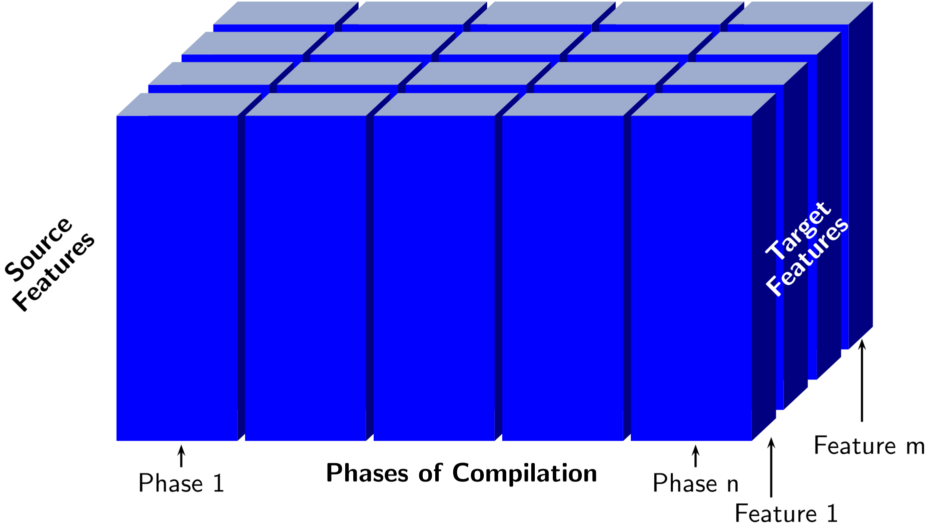
In Search of Modularity in Compilation



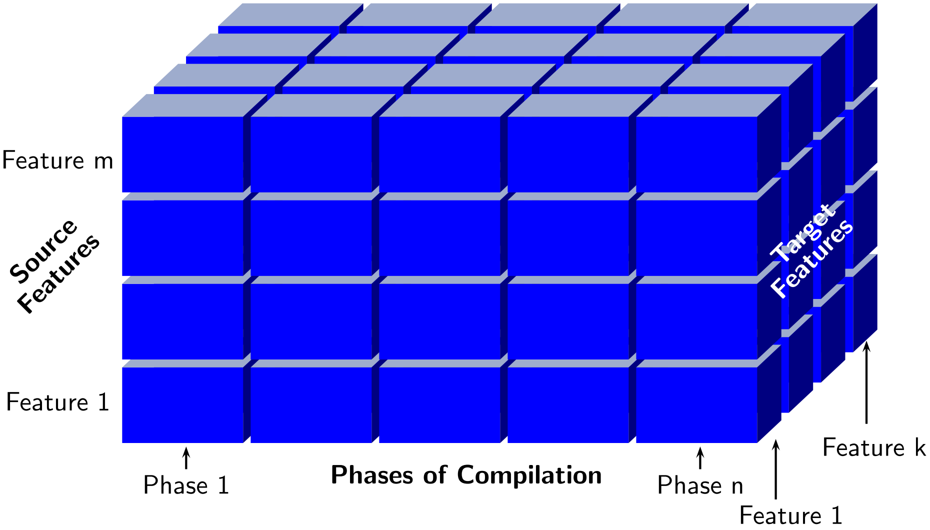
In Search of Modularity in Compilation



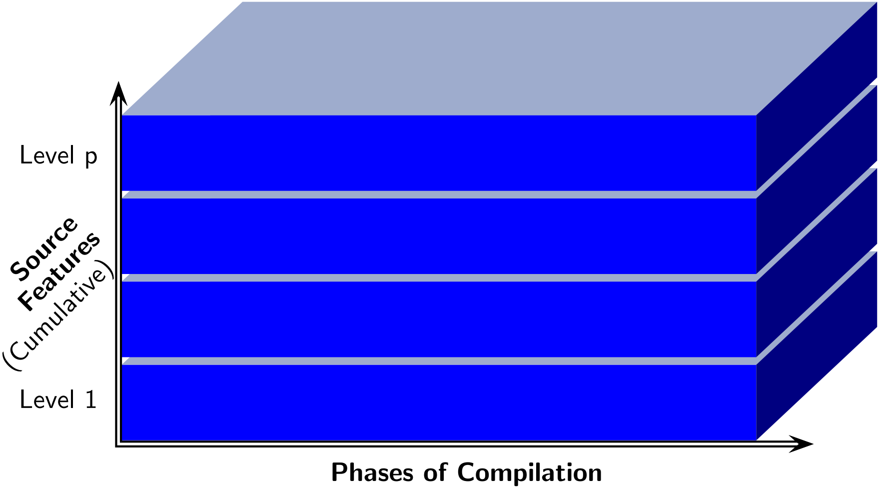
In Search of Modularity in Compilation



In Search of Modularity in Compilation



In Search of Modularity in Compilation



Language Increments

Assignments with simple RHS

Level 1

Language Increments

Arithmetic Expressions

Assignments with simple RHS

Level 1

Level 2

Language Increments

Comparison and Logical Expressions

Arithmetic Expressions

Assignments with simple RHS

Level 1

Level 2

Level 3

Language Increments

Control Structures

Comparison and Logical Expressions

Arithmetic Expressions

Assignments with simple RHS

Level 1

Level 2

Level 3

Level 4

Language Increments

Function Calls (including recursion)

Control Structures

Comparison and Logical Expressions

Arithmetic Expressions

Assignments with simple RHS

Level 1

Level 2

Level 3

Level 4

Level 5

Language Increments

Pointers and Arrays

Function Calls (including recursion)

Control Structures

Comparison and Logical Expressions

Arithmetic Expressions

Assignments with simple RHS

Level 1

Level 2

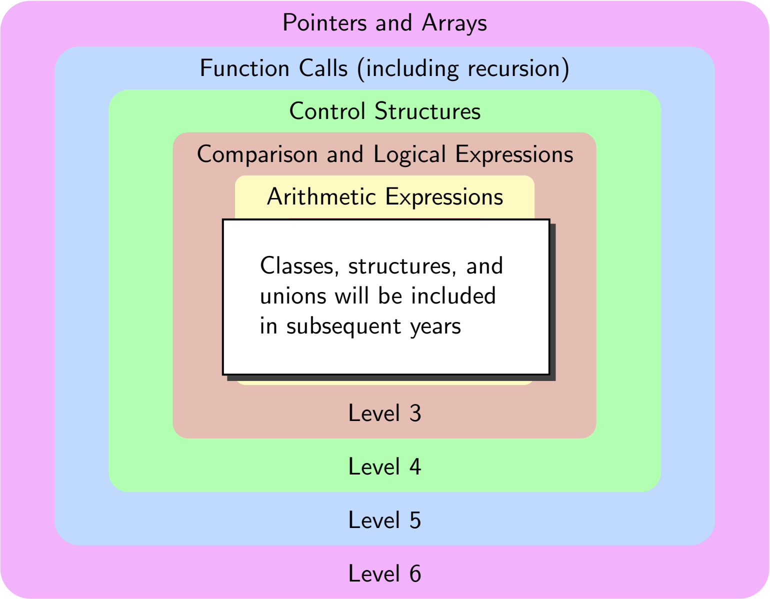
Level 3

Level 4

Level 5

Level 6

Language Increments



SCLP Increments

Pointers and Arrays

Function Calls

Control Structures

Comparison and Logical Expressions

Arithmetic Expressions

Assignments with simple RHS

Level 1

Level 2

Level 3

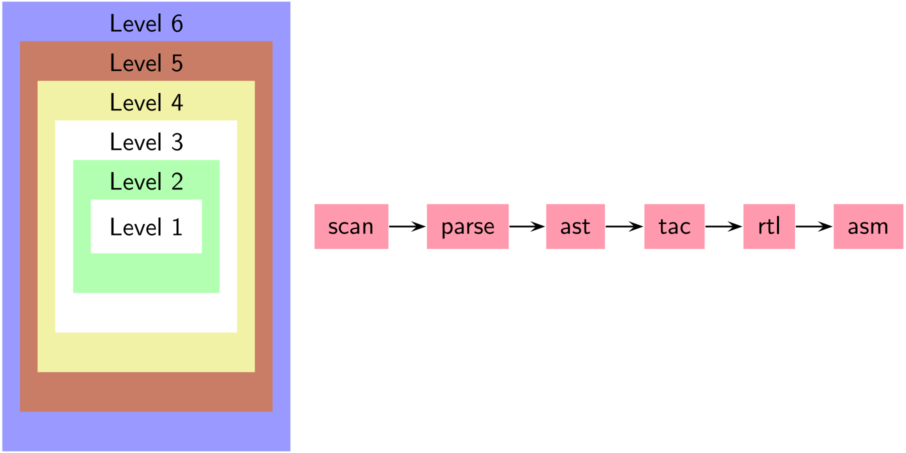
Level 4

Level 5

Level 6

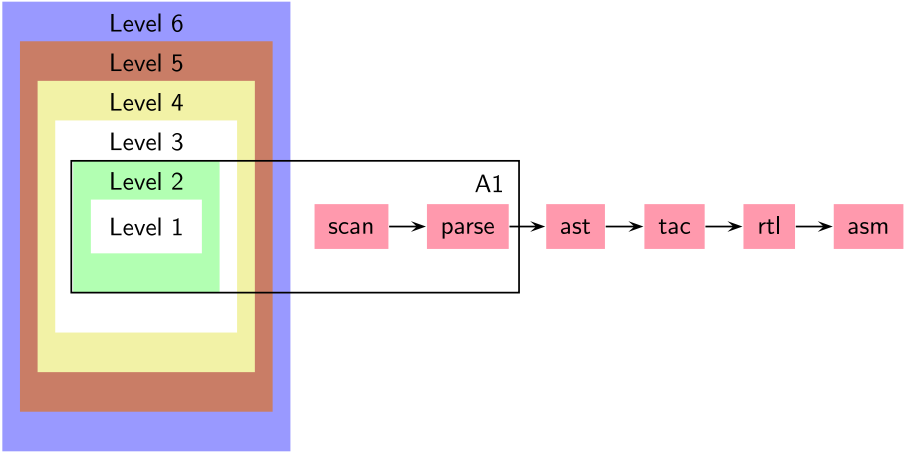
Proposed Assignment Plan: Incremental Construction

A series of assignments; each assignment builds on the previous assignment



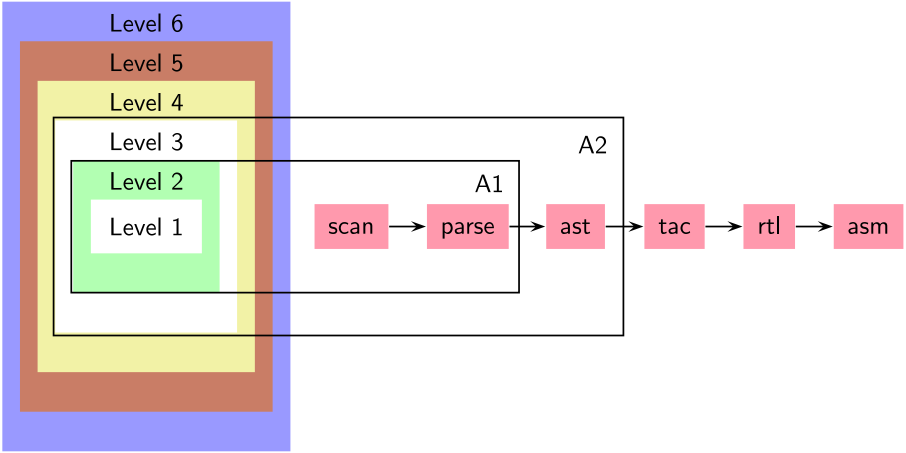
Proposed Assignment Plan: Incremental Construction

A series of assignments; each assignment builds on the previous assignment



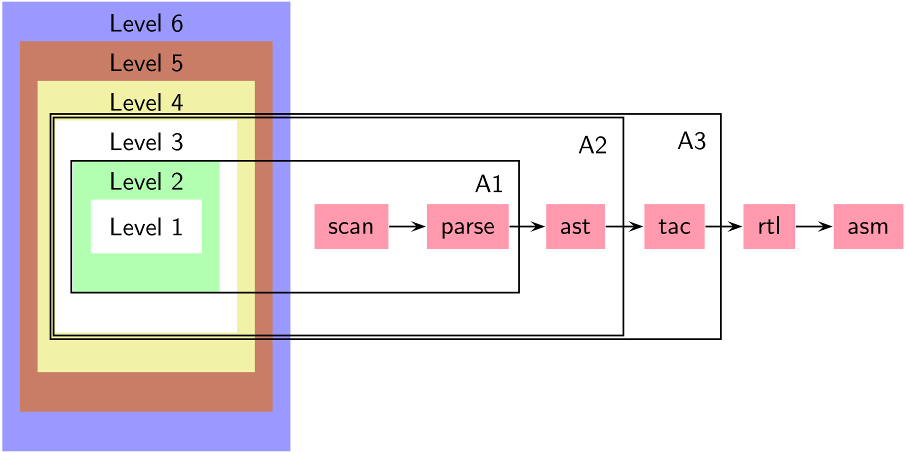
Proposed Assignment Plan: Incremental Construction

A series of assignments; each assignment builds on the previous assignment



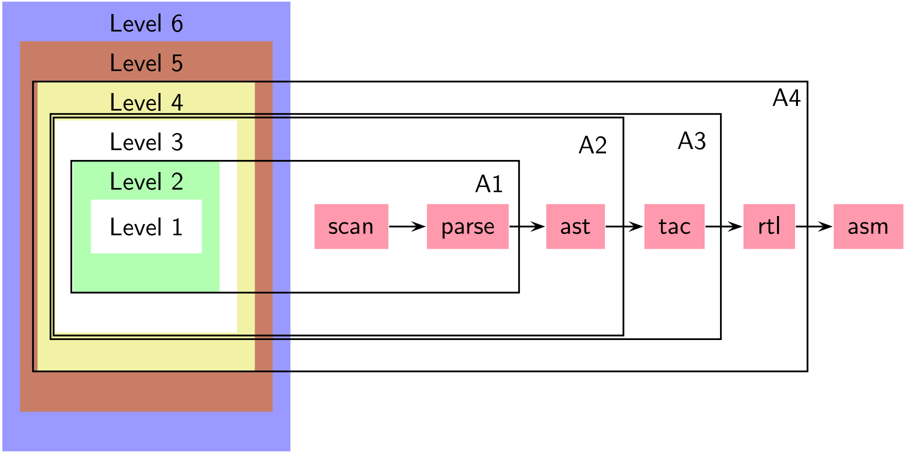
Proposed Assignment Plan: Incremental Construction

A series of assignments; each assignment builds on the previous assignment



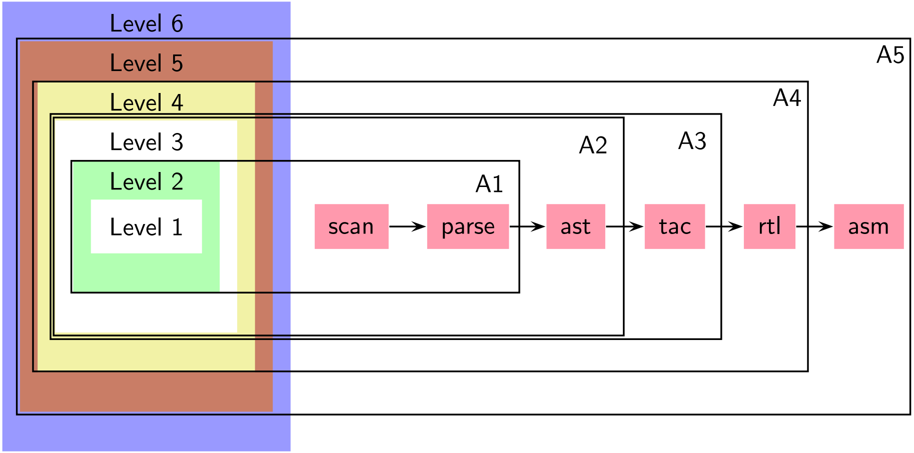
Proposed Assignment Plan: Incremental Construction

A series of assignments; each assignment builds on the previous assignment



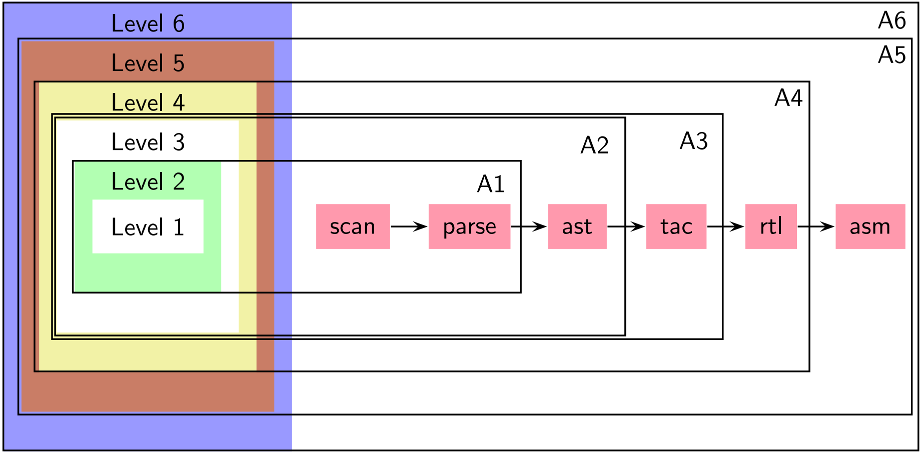
Proposed Assignment Plan: Incremental Construction

A series of assignments; each assignment builds on the previous assignment



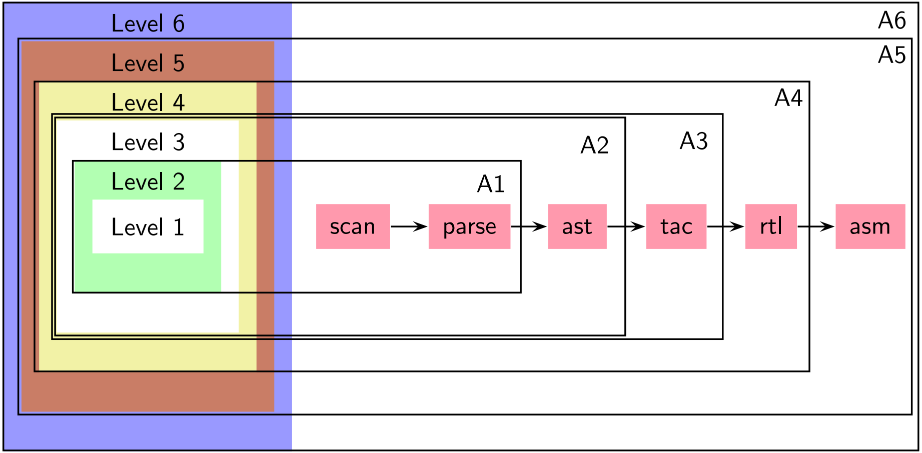
Proposed Assignment Plan: Incremental Construction

A series of assignments; each assignment builds on the previous assignment



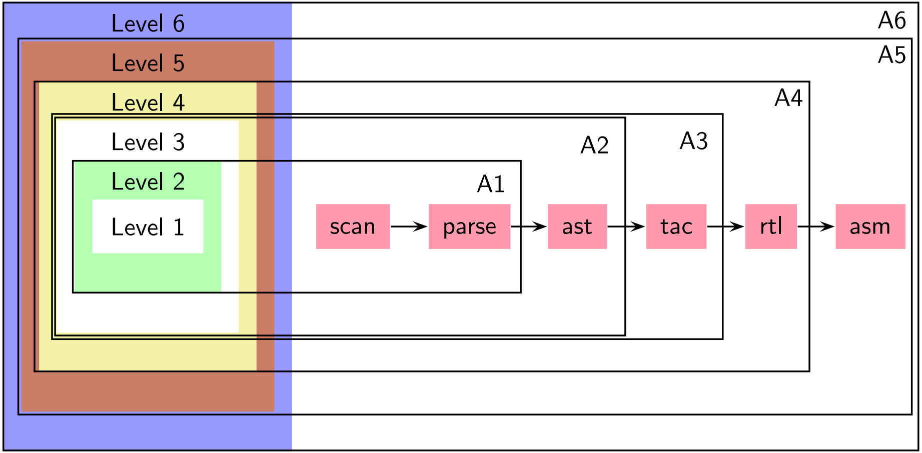
Proposed Assignment Plan: Incremental Construction

A series of assignments; each assignment builds on the previous assignment



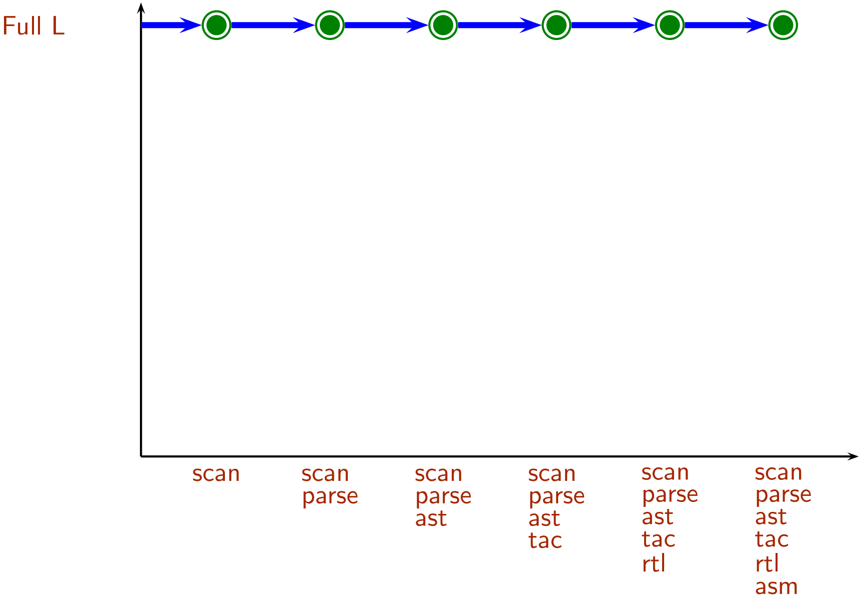
Proposed Assignment Plan: Incremental Construction

A series of assignments; each assignment builds on the previous assignment

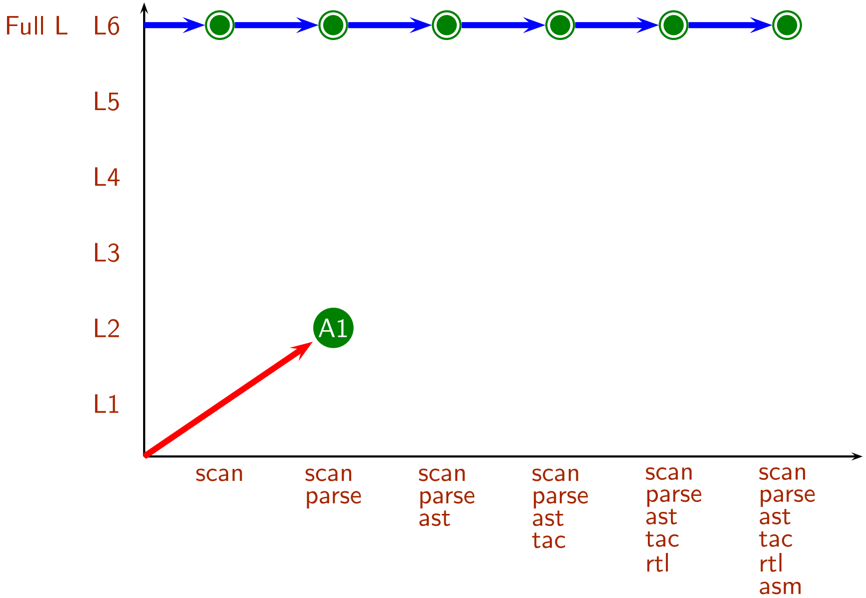


A6 is optional

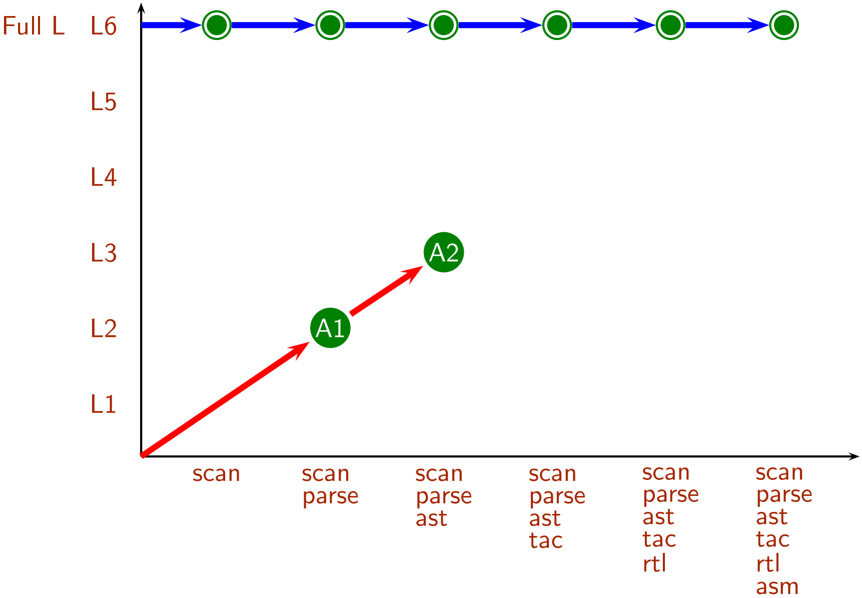
Conventional Increments Vs. Proposed Increments



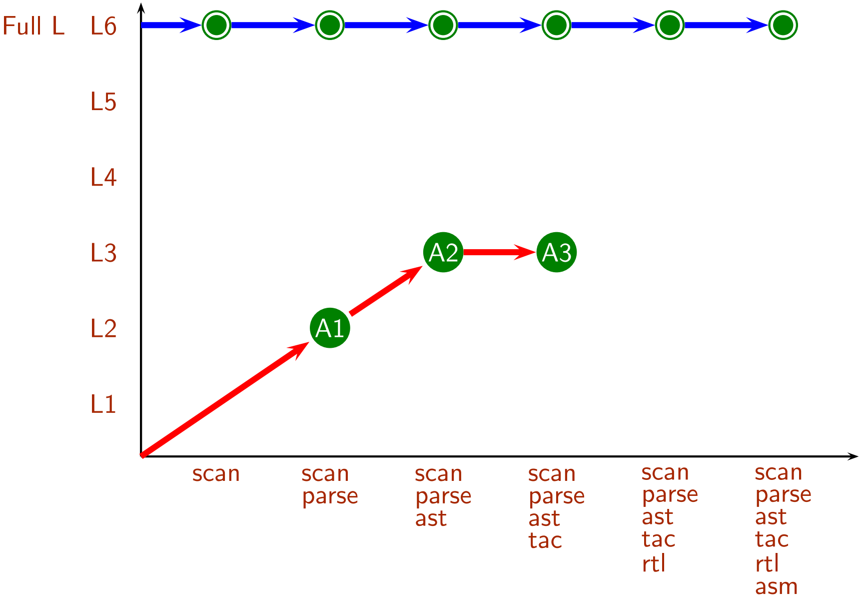
Conventional Increments Vs. Proposed Increments



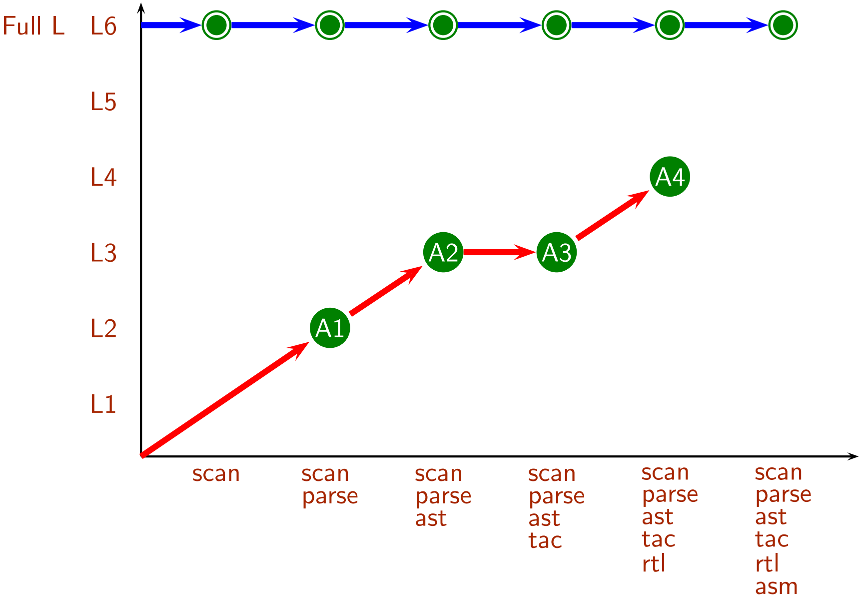
Conventional Increments Vs. Proposed Increments



Conventional Increments Vs. Proposed Increments



Conventional Increments Vs. Proposed Increments



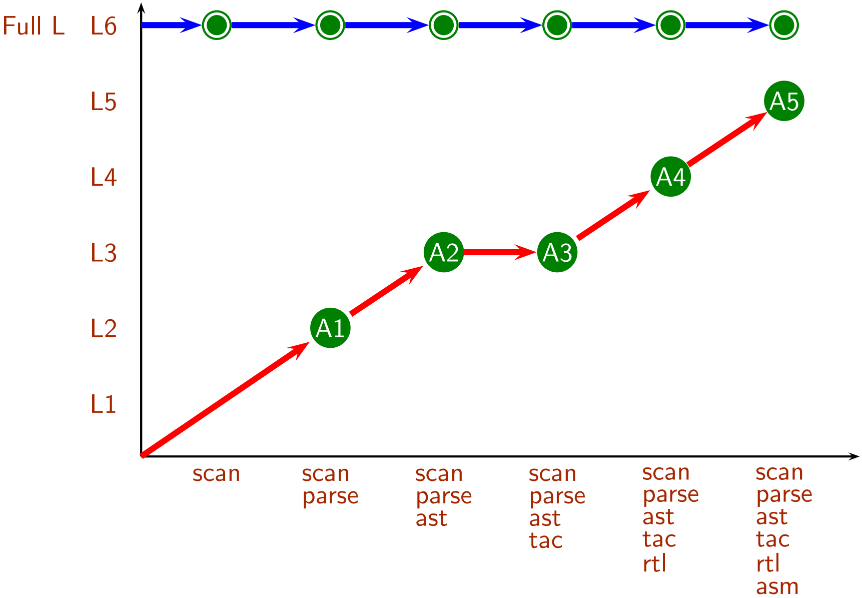
IIT Bombay
cs302: Implementation of Programming Languages

Topic:

Compilation Overview

Section:

Conventional Increments Vs. Proposed Increments



IIT Bombay
cs302: Implementation of Programming Languages

Topic:

Compilation Overview

Section:

Outline

The Big Picture

Course Plan

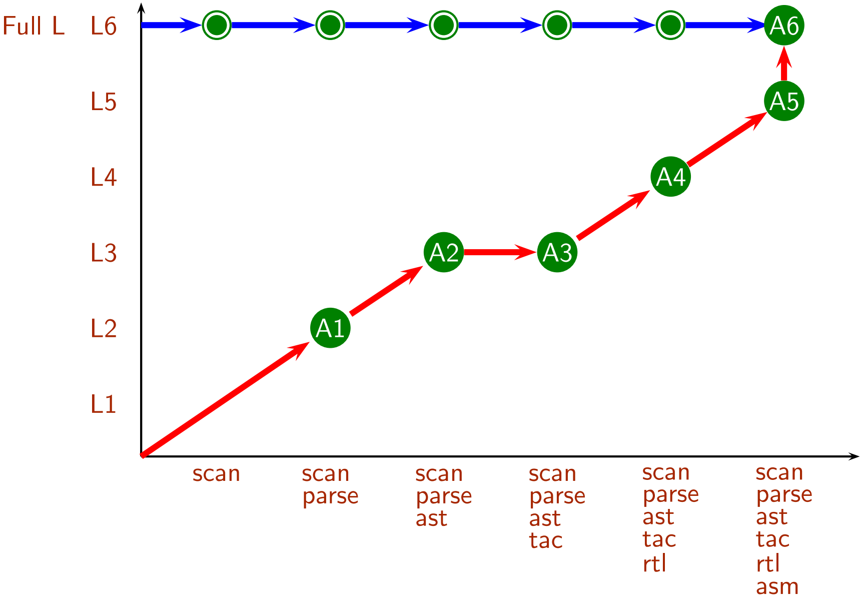
Expectation Management

An Overview of Compilation Phases

Compilation Models

Incremental Construction of Compilers

Conventional Increments Vs. Proposed Increments



Conventional Increments Vs. Proposed Increments